面向 Pending 交易的 DeFi 价格操纵实时预警系统：Cash-Flow Graph 增强与可降级实现

姓名：王牧天

学号：LJ2606114

2026 年 5 月 21 日

摘要

去中心化金融（Decentralized Finance, DeFi）协议中的价格操纵攻击通常在单笔或少数几笔以太坊交易内完成，现有检测方案在攻击上链后才告警，损失已不可逆。本文提出一种面向 pending 交易的 DeFi 价格操纵实时预警系统，将检测时机从事后前移至出块之前。在建模上，本文在已有 Cash-Flow Graph 工作的 Address/Asset 节点和 Transfer 边基础上，设计 Protocol 节点、Call 边和 Context 边的增量异构图模型（四类节点、三类边），以编码协议角色、函数级语义和时序上下文。检测层采用"规则基线 + GraphSAGE"双层体系，并实现模型不可用时的真实规则退化；告警层已生成可回放的事件级内部证据引用，路径级 EvidenceRef 统一规范作为生产化目标。在 Provider-only 受限环境下，系统以 Uniswap v3、Aave v3、Compound III、Chainlink 为目标协议完成端到端原型实现，并在 110 条唯一评分链路上验证 trace/graph/risk/alert/本地检索链路一致性和三级降级协议。完整 trace 环境下的 GraphSAGE 训练、Call/Context 边效果验证和生产级延迟评估属于进一步工作。

关键词：DeFi 安全；价格操纵检测；图神经网络；Mempool 监控；实时预警

1 绪论

1.1 研究背景与问题提出

去中心化金融（DeFi）是以太坊等智能合约平台上最重要的应用类别之一，涵盖去中心化交易所（DEX）、借贷市场、衍生品协议、收益聚合器和真实世界资产（RWA）等场景。DefiLlama 的 2025 年研究报告显示，DeFi 的多类子市场仍保持数百亿美元量级的锁仓规模；SlowMist 2025 年度安全报告则显示，DeFi 是当年被攻击频次最高的项目类型，126 起事件约占全年事件总数的 63%，造成约 6.49 亿美元损失[1][2]。这说明 DeFi 的资金承载能力与安全风险仍然高度集中。

价格操纵攻击的典型特征是：攻击者在一笔或少数几笔交易内通过闪电贷获取巨额资金，操纵 AMM 池的现货价格或扭曲预言机报价，再利用被操纵的价格在借贷协议、衍生品协议或流动性池中借出超额资产、触发非正常清算或完成套利。bZx（2020，合计损失接近 100 万美元）、Cream Finance（2021，损失约 1.3 亿美元）、Mango Markets（2022，损失约 1.16 亿美元）和 Euler Finance（2023，损失约 1.97 亿美元）等事件均体现了跨协议价格依赖被利用后的高杠杆风险[22][5][4][23]。

以太坊的交易生命周期以约 12 秒的 slot 为基本节奏。交易首先被广播至 mempool（交易池），由区块提议者（proposer）和构建者（builder）选择打包进区块，区块经共识确认后交易才最终上链。现有 DeFi 安全检测系统——无论是基于规则的 DeFiRanger[6]、基于图神经网络的 DeFiGuard[7]、还是 CertiK Skynet[8] 等商业服务——

主要基于已上链交易或事后取证进行分析。从攻击交易被打包到告警生成之间即使只间隔数秒，资金状态也已经发生不可逆变化。

核心问题：能否在交易仍处于 pending 阶段（尚未被包含进区块）时完成风险评分和证据化告警，从而把安全监控从“上链后发现”前移到“出块前感知”？

实现这一目标面临三类特有挑战：（1）**时间约束**——从监听 pending 交易到输出告警，全链路必须控制在秒级预算内，否则只能成为事后分析；（2）**信息不完整**——pending 交易可能被替换（相同 nonce 的新交易挤出旧交易）、被丢弃（低 gas 价格被更高出价挤出）、被重排序（MEV bundle 内部重组），部分交易来自私有 mempool 不可见，且未上链 trace 模拟的结果与最终上链状态之间无法保证一致；（3）**安全边界**——误报代价是运营负担，漏报代价是资金损失，系统必须支持可调阈值、人工复核和可审计证据，自动链上防御在权限与治理流程未明确前不进入默认范围。

1.2 研究意义

安全意义：本系统将检测时机从事后（攻击已上链、损失已发生）提前至事中（交易在 mempool、尚未出块），为协议运营方和用户争取更早的风险感知窗口。在实际运营中，预警更现实的作用是尽早触发人工复核、后续区块级处置、协议暂停讨论和损失控制，而不是保证在同一区块内阻止攻击。

技术意义：pending 交易的不稳定性和 trace 信息的不完整性为检测带来了已有事后方法未面对的挑战。本文尝试将检测时机从已上链阶段前移至 pending 阶段，完成了 Provider-only 原型验证，同时已实现完整 trace 环境下所需的 Cash-Flow Graph 增强

代码入口。完整 trace 环境下的端到端实时推理验证仍需要本地节点支撑，属于进一步工作。

实践意义：通过多层降级协议（trace 失败 → 轻特征包；上下文超时 → tx-only 图；模型不可用 → 规则基线）提高检测链路的连续性，为其他延迟敏感的链上分析系统提供可复用的设计模式；但当前单 Provider 数据源仍是明确的可用性单点。

1.3 国内外研究现状

DeFi 安全检测领域的主要工作可分为三类。

基于规则/语义的检测：DeFiRanger[6] 通过对 3.5 亿笔以太坊交易的分析，定义了 7 类攻击模式（包括价格操纵、闪电贷攻击、预言机操纵等），使用语义规则匹配和异常利润路径分析检出 432 起真实攻击。其优点是可解释性强，但规则维护成本高，且难以发现未知攻击模式。

基于图神经网络的检测：DeFiGuard[7] 提出将 DeFi 交易建模为 Cash-Flow Graph，并使用 GraphSAGE[9] 进行归纳式节点嵌入和二分类检测，在 208/2080 数据集上达到 93.25% Accuracy 和 96.18% AUC，端到端分类耗时 0.892–5.317 秒。STA-GT[10] 和 FraudGT[21] 进一步说明时序图 Transformer 和带边属性的图 Transformer 可用于更一般的金融欺诈检测，但其任务设定并非 pending 阶段 DeFi 价格操纵预警。这些方法的共同局限是检测输入来自已观察或已确认交易记录，而不是面向未上链 pending 交易的 `debug_traceCall` / `trace_call` 模拟结果。

商业安全服务：CertiK Skynet[8]、SlowMist MistTrack[2] 和 Chainalysis[11] 提供不同程度的链上监控和威胁情报服务，但其方法论细节未公开，且检测时效性通常为分钟级或更长。

现有方法的三个共性局限——（1）事后检测；（2）缺乏 pending 阶段 trace 能力；（3）告警缺乏可复核的证据引用——构成了本文的研究动机。

1.4 研究内容与主要贡献

贡献一（pending 阶段检测）：本文尝试将检测时机从事后（已上链）前移至 pending 阶段（mempool），设计端到端延迟目标为 P95 < 3s、P99 < 8s。与现有事后检测方法的根本区别在于：本系统必须应对 pending 交易的不稳定性——交易可能被替换（nonce 冲突）、被丢弃（低 gas 被挤出）、被重排序（MEV bundle 内部重排）、来自不可见 mempool（私有 relay）。本文在第 3 章和第 4 章中分别给出了协议层和算法层的应对方案。当前原型验证了管线架构和降级协议的可行性，完整 trace 环境下的延迟实测属于进一步工作。

贡献二（Cash-Flow Graph 增量建模）：在 DeFiGuard 的 Address/Asset 节点和 Transfer 边基础上，设计 **Protocol 节点**（编码协议角色、审计状态、治理权限、pause 能力）、**Context 边**（编码 5 分钟地址上下文和 1 小时协议上下文的时序关系）和 **Call 边**（编码 call depth、selector、call_type 的函数级语义），构成四类节点、三类边的异构图。Protocol 节点使得模型能区分同一地址在不同协议中的不同角色；Context 边使得模型在仅看到单笔交易图时也能感知近期异常模式。当前原型已实现 Protocol

节点构建、Call 边轻量语义解析、Context 窗口聚合与时序边生成，并保留 trace/context 不足时的降级路径，完整 Call/Context 边效果验证待 trace 环境就绪后执行。与 DeFiGuard 的详细对比见第 2.2.3 节。

贡献三（双层检测与证据规范）：构建"规则基线 + GraphSAGE"双层检测体系——当前原型中规则基线作为主工作评分器和模型退化时的安全兜底，GraphSAGE 作为完整 trace 数据就绪后的在线主模型设计——并提出统一的 EvidenceRef 三类证据引用目标规范（GRAPH/DOC/LABEL）。当前实现已生成 risk/graph/trace/label 内部引用并检查引用对象存在性；路径级 GRAPH 引用和统一格式强制校验是生产化待完成项。

贡献四（系统实现与验证）：以 Uniswap v3、Aave v3、Compound III、Chainlink 四类协议为白名单，完成端到端系统原型。Provider-only 管线完成 1,015 个 replay 工件审计，并在 110 条唯一评分链路上通过 9 项 replay 检查（pending_replay、trace_audit、graph_replay、risk_replay、alert_replay、rag_explanation、console_view、offline_eval、failure_drill），故障注入验证了三级降级协议的健壮性。标注数据管道就绪（7 笔 exploit 候选 + 50 笔正常候选，覆盖 contract_call/erc20_transfer/erc20_approve/simple_transfer 四类交易模式），并已补齐 Context 窗口构建、图级标签导出、GraphSAGE 训练与推理入口；但因当前环境不依赖本地归档节点和付费 trace API，GraphSAGE 权重训练待 trace 数据和 promoted 图级标签就绪后执行。实现规模和配置清单移至附录说明。

1.5 论文组织结构

本文余下部分按如下方式组织：第 2 章回顾相关工作与技术背景；第 3 章阐述系统架构设计（设计决策与接口协议）；第 4 章详细描述核心算法方法（Cash-Flow Graph 构建、规则基线、GraphSAGE、RAG 证据解释）；第 5 章讨论系统实现细节与工程取舍；第 6 章报告已验证结果与完整 trace 环境下的评估设计；第 7 章讨论系统局限性、权衡与发展方向；第 8 章总结全文并展望未来工作。

2 相关工作与技术背景

2.1 DeFi 价格操纵攻击分类学

理解攻击模式是设计检测系统的前提。本节基于已有安全报告和学术文献，将 DeFi 价格操纵攻击归纳为以下类别。

Oracle 操纵：攻击者通过操纵链上价格预言机的输入源来扭曲报价，进而在依赖该报价的借贷协议中获利。典型手法包括：（1）TWAP 操纵——在短时间内通过大量 swap 交易扭曲基于时间加权平均价格的预言机，使 TWAP 偏离真实市场价格；（2）闪电贷价格扭曲——使用闪电贷放大操纵规模，在单笔交易内完成"借款→扭曲→套利→还款"；（3）陈旧喂价利用——利用部分预言机更新频率低的特点，在两次更新之间制造价差。Cream Finance（2021，损失约 1.3 亿美元）和 Inverse Finance（2022，损失约 1560 万美元）攻击均涉及 Oracle 操纵[5][12]。

AMM 流动性攻击：攻击者利用自动做市商（AMM）的流动性分布特征实施价格操纵。典型手法包括：（1）薄池交易——针对低流动性或集中流动性池（如 Uniswap V3 的窄 tick 区间）发起大额 swap，大幅移动价格后在其他协议中变现；（2）闪电贷价格泵——使用闪电贷资金瞬间放大交易规模，在池中制造极端价格后在其他协议中获利；（3）三明治辅助操纵——通过在前置和后置交易中安排关联 swap，人为制造价格波动路径。Mango Markets（2022，损失约 1.16 亿美元）攻击的核心机制即涉及对 MNGO 代币的薄池价格操纵[4]。

跨协议清算级联：攻击者通过价格操纵触发借贷协议中的清算条件，导致抵押品被强制折价出售，进而引发跨协议的价格传导和连锁清算。此类攻击往往涉及多个借贷市场，且坏账风险具有传导性。

攻击共性特征：上述攻击的共同结构特征是——（1）单笔交易内完成多协议间的资金流动；（2）存在从"价格操纵点"到"套利变现点"的利润路径；（3）攻击者净资本投入接近于零（通常由闪电贷提供全部资金）；（4）利润路径的方向与正常套利路径相反（攻击者输入操纵，正常交易者输入信息）。

2.2 现有检测方法与系统

2.2.1 基于规则/语义的方法

DeFiRanger[6] 是目前规模最大的 DeFi 攻击规则检测系统。它定义了 7 类攻击模式（价格操纵、闪电贷攻击、预言机操纵、重入攻击、访问控制漏洞、治理攻击、MEV 利

用），在 3.5 亿笔以太坊交易中检出 432 起真实攻击。其核心方法是：（1）将交易执行 trace 解析为资金流图；（2）在图结构上匹配预定义的攻击语义模式（如"闪电贷 borrow → swap → deposit → borrow → repay → swap → repay"路径）；（3）基于"异常利润"信号过滤误报。DeFiRanger 的优点是精确率高、可解释性强，但其规则集依赖已知攻击模式的事后总结，对未知攻击变种的覆盖不足，且需要维护不断增长的模式库。

2.2.2 基于图神经网络的方法

DeFiGuard[7] 将 DeFi 交易检测首次形式化为 Cash-Flow Graph 上的节点分类问题。它将地址和资产建模为两类节点，将代币转移建模为 Transfer 边，使用两层 GraphSAGE 对节点进行归纳式嵌入，输出每个节点的"操纵概率"。在包含 208 起攻击和 2080 笔正常交易的数据集上，DeFiGuard 达到 93.25% Accuracy 和 96.18% AUC，推理延迟 0.892–5.317 秒。

STA-GT[10] 面向金融交易欺诈检测引入 Spatial-Temporal-Aware Graph Transformer，通过时间编码和 Transformer 模块捕获交易图中的空间-时间依赖。该方法说明时序图 Transformer 可用于更复杂的交易欺诈模式建模，但其任务设定并非 DeFi 价格操纵检测；因此本文仅将其作为冷路径二级复核的相关架构参考，而不把其指标直接等同于 DeFi PMA 任务结果。

2.2.3 与 DeFiGuard 的逐点对比

DeFiGuard 是与本文方法最接近的已有工作。下表列出本文在建模上的增量：

维度	DeFiGuard	本文	增量贡献说明
检测时机	已上链交易（事后）	pending 交易（出块前）	需处理 trace 模拟不确定性、超时降级、mempool 可见性问题
节点类型	Address, Asset	Address, Asset, **Protocol, Transaction**	Protocol 节点编码协议角色（pool/lending/oracle/router）、审计状态、治理权限（pause 能力），使模型能区分同一地址跨协议的异构行为
边类型	Transfer	Transfer, **Call**, **Context**	Call 边编码函数级语义（selector、call depth、delegatecall 标识）；Context 边编码时序关系（5min 地址窗口、1h 协议窗口），使模型在单交易图中也能感知近期异常
上下文窗口	无	三窗并行：tx-only / 5m-address / 1h-protocol	上下文窗口异步回填，超时不阻塞主图推理；缺失特征显式标注 `feature_missing_flags`，保证训练/推理特征语义一致
特征工程	account_type, in/out degree	增加协议角色、function selector、oracle 读写、滑点、MEV 相对位置	面向在线部署增加了 trace 失败时的降级特征（仅从 raw_tx calldata 可解析的字段）
退化机制	未提及	规则基线作为 GraphSAGE 退化的安全兜底	规则基线不依赖训练数据分布，在模型版本不匹配或服务不可达时自动接管，保证 100% 交易有分数输出
证据与	无	EvidenceRef 三类证据 + 异步 RAG	当前实现事件级内部引用；路径级强校验待生产化

维度	DeFiGuard	本文	增量贡献说明
解释			

Protocol 节点和 Context 边的引入改变了图的拓扑结构。例如，一笔交易同时触达 Uniswap V3 Pool 和 Aave V3 Pool，在 DeFiGuard 的图中两者只是两个 Asset 节点；在本文的图中，两个 Protocol 节点分别携带了各自协议的审计状态、治理权限和 pool 类型，使得模型可以学习"跨协议价格操纵路径"这一模式。类似地，Context 边使得一笔在 5 分钟窗口内被同一地址连续发起多次 swap 的交易，其图结构与孤立交易有本质不同。

2.2.4 其他相关工作

商业服务：CertiK Skynet[8] 提供基于实时链上数据的风险评分和攻击预警，但其方法细节未公开。SlowMist MistTrack[2] 维护了 4 亿+ 地址标签和 50 万+ 威胁情报条目，主要用于事后溯源而非实时检测。Chainalysis[11] 侧重于合规和反洗钱（AML）场景。

以太坊 MEV 研究：Daian 等人在"Flash Boys 2.0"[13] 中首次系统分析了以太坊交易重排序和抢跑行为，揭示了 pending 交易被 MEV 机器人操纵的普遍性。这一发现为本文关注 pending 阶段检测提供了重要动机——不仅攻击者可以在 pending 阶段发起操纵，MEV 基础设施（如 Flashbots[14]）的存在也意味着部分交易路径对公开 mempool 不可见。

2.3 以太坊 Mempool 与 Trace 技术

2.3.1 Mempool 可见性

以太坊中，用户提交的交易首先进入节点的交易池（txpool 或 mempool），等待被区块提议者选中打包。公开 mempool 中的交易可以通过 `eth_subscribe`（WebSocket）或 `txpool_content`（HTTP 轮询）获取。然而，以下因素使得 mempool 视角不完整：

- **私有 mempool**：部分交易通过 Flashbots Protect 等私有 RPC 端点提交，绕过公开 mempool 直接到达区块提议者，在公开 mempool 中不可见。
- **MEV 中继（relay）**：MEV-Boost 生态系统中，交易和 bundle 通过中继在提议者和构建者之间传递，这部分流量不经过公开 mempool。
- **交易替换**：相同 nonce 的新交易可以替换（replace）或加速（speed up）已在 mempool 中的交易，导致模拟的交易内容与最终上链内容不一致。
- **交易丢弃**：低 gas 价格的 pending 交易在网络拥堵时可能从 mempool 中被驱逐。

本文的处理策略是：承认 mempool 信息不完整性，系统设计为"尽最大努力检测可见交易"，而非"保证所有交易全覆盖"。私有 mempool 交易和 MEV bundle 的检测需要与 Flashbots 等基础设施的深度集成，属于当前范围外的进一步工作。

2.3.2 Trace 模拟技术

对已上链交易，可以使用 `debug_traceTransaction`（Geth）或 `trace_transaction`（Erigon）获取完整的执行痕迹，包括内部调用、状态变化和事

件日志。对 pending 交易，不能使用这些方法（pending 交易还没有区块位置和最终状态），而须使用：

- **Geth `debug_traceCall`**：在指定状态上下文（如 `pending` 状态）上模拟执行一笔交易，返回调用树和状态变化。支持 `state_overrides` 和 `block_overrides` 参数，允许在模拟中修改特定地址的余额、代码或存储，或修改区块参数（如时间戳、base fee）。
- **Erigon `trace_call`**：功能类似于 `debug_traceCall`，但返回格式不同，支持 `trace` 和 `stateDiff` 两种模式。

未上链 trace 模拟的两个核心局限：

1. **模拟结果不等同于上链结果**：pending 交易可能与其他交易产生状态冲突（如使用相同的 nonce），或在实际执行时因状态变化而得到不同的结果。系统必须记录 `trace_source` 和 `trace_status` 以区分"模拟成功"和"上链确认"。
2. **模拟延迟不可控**：复杂交易（多协议交互、多层级调用）的 trace 模拟可能超过 1.5 秒，超出热路径预算。系统必须设置硬超时并实施降级。

2.4 图神经网络在图异常检测中的应用

2.4.1 GraphSAGE

GraphSAGE（Graph Sample and Aggregate）[9] 是一种归纳式图表示学习框架。与 GCN（需要在训练时知道完整的图拉普拉斯矩阵，对新节点需要重新训练）不同，

GraphSAGE 学习的是聚合函数而非固定嵌入。对于给定节点 v，第 k 层的聚合-更新操作定义为：

$$h_v^k=\sigma\left(W^k\cdot \mathrm{CONCAT}\left(h_v^{k-1},\mathrm{AGG}_k\left(\left\{h_u^{k-1}:u\in N(v)\right\}\right)\right)\right)$$

GraphSAGE 支持的聚合函数包括 Mean、LSTM 和 Pooling。Mean Aggregator 计算邻居特征的元素均值，是归纳式设置中最常用的选择。

在 DeFi 安全检测场景中，GraphSAGE 的归纳式特性具有关键优势：（1）新部署的协议合约地址在训练时未知，可以在推理时直接生成嵌入；（2）攻击者频繁更换地址（一次性地址），归纳式推理使其无法通过地址更换规避检测。

2.4.2 其他 GNN 架构对比

架构	归纳式	支持异构图	推理延迟	在 DeFi PMA 上的已知表现
GCN[15]	否（直推式）	需改造	低	Accuracy ~87%
GAT[16]	是	需改造	中	Accuracy ~90%
GIN[17]	是	需改造	低	Accuracy ~89%
GraphSAGE[9]	是	可通过框架关系封装适配	低	Accuracy 93.25%, AUC 96.18%[7]
Graph Transformer[18]	是	支持	高	可建模空间-时间依赖；本文仅作冷路径参考[10]

注：表中 "在 DeFi PMA 上的已知表现" 数据来自 DeFiGuard[7] 的横向对比实验。

架构	归纳式	支持异构图	推理延迟	在 DeFi PMA 上的已知表现
GCN/GAT/GIN/MLP 的原始论文[15][16][17] 并未直接评估 DeFi 价格操纵检测任务，此处引用的是 DeFiGuard 作者在其数据集上的复现结果。				

基于上述对比，本文在完整 trace 部署方案中选择 GraphSAGE 作为在线主模型，保留 Graph Transformer 作为 top 1-5% 高风险交易的二级复核选项（不进入一期热路径）。当前原型已补齐两层 GraphSAGE 模型定义、训练入口、推理入口和模型 manifest/version 协议；但由于缺少 trace-backed 图级标签数据和当前环境未安装 PyTorch/PyG，尚未训练或发布可用权重。

2.5 RAG 在安全分析中的应用

检索增强生成（Retrieval-Augmented Generation, RAG）[19] 是一种将外部知识检索与大型语言模型（LLM）生成相结合的技术。其核心流程为：（1）将用户查询编码为向量；（2）从外部知识库中检索相关文档；（3）将检索到的文档与用户查询拼接后送入 LLM 生成回答。

在安全分析领域，RAG 的核心价值是"证据约束"——LLM 的回答被限定在已检索到的权威文档范围内，而非自由生成。这一特性对安全告警尤其重要：告警说明中的每个断言都应该是可追溯的（"根据某协议文档第 X 节的描述……"），而不是模型凭空生成的。

本文的 RAG 模块在冷路径（异步）运行，不参与热路径风险评分——这一设计决策的理由是：LLM 推理延迟（2-5 秒）不可控，若进入热路径会导致告警在出块后才到达，丧失预警意义。RAG 的输出仅用于充实告警的解释性内容，不直接影响风险分数的计算。

3 系统设计

> 本章阐述设计决策和接口协议——即系统"为何这样设计"和各层之间的契约。具体实现细节（技术栈选型、模块结构、配置管理等）见第 5 章。

3.1 设计约束与架构决策

3.1.1 核心约束

本系统面临三个刚性约束：

时间约束（C1）：以太坊 slot 约 12 秒。系统检测必须在交易被包含进区块前完成，否则预警意义丧失。考虑到网络延迟、节点同步抖动和操作人员反应时间，端到端延迟预算分配为 P95 < 3 秒、P99 < 8 秒。

信息不完整约束（C2）：pending 交易可能被替换（相同 nonce 的新交易）、被丢弃（低 gas 价格挤出）、被重排序（MEV bundle 内部重组），部分交易来自私有 mempool 完全不可见。此外，未上链 trace 模拟（`debug_traceCall`）的结果与最终

上链状态之间没有保证一致性——模拟时使用的"pending"状态与区块实际打包时的状态可能不同。

安全边界约束（C3）：误报代价 = 运营负担（分析员时间、不必要的协议暂停讨论成本、可能的链上防御交易 gas 成本）；漏报代价 = 资金损失（单次攻击损失可达千万至亿美元级别）。系统必须在两者间取得可调节的平衡。自动链上处置（如自动发送暂停交易）默认为禁区——涉及智能合约访问权限和治理流程，在权限模型和授权流程未明确前不应实施。

3.1.2 关键架构决策及理由

决策点	选择	排除的备选方案	理由
D1: 热/冷路径分离	评分链路同步，证据解释异步	全同步（RAG 入热路径）	RAG 涉及 LLM 推理（2-5s），若入热路径会导致告警在出块后才到达，丧失预警意义
D2: 数据源优先级	本地节点为主，Provider（Alchemy）为备	纯 Provider / 纯本地节点	Provider 仅能看到自身 mempool 视角，不完整；高级 Trace/Debug API 往往在付费层。纯本地节点运维成本高、无冗余。两者结合兼顾完整性和可用性
D3: 主模型选择	GraphSAGE（2 层归纳式）	GCN/GAT/GIN（性能未经验证）、Graph Transformer（延迟过高）	在 DeFiGuard[7] 同类任务上已验证有效（AUC 96.18%）；归纳式推理允许新地址即到即推；本文将完整 trace 环境下的推理延迟目标设为 < 50ms
D4: 图	单笔交易为主图 +	全局时序图 / 纯单交易图	纯单交易图缺失时间上下文；全局时

决策点	选择	排除的备选方案	理由
粒度	上下文窗口为辅		序图构建和推理开销超出热路径预算。单交易主图 + 异步回填上下文窗口是折中方案
D5: 处置策略	默认人工批准后执行	自动链上执行	误报可能造成的协议暂停损失超过单次攻击漏报；自动执行涉及合约权限委托和治理审批，需额外法律/治理框架

3.2 管道分层与接口协议

系统分为五层，层间通过版本化事件接口解耦。每层只定义输入/输出契约和失败降级协议，不规定具体实现方式。层与层之间是单向依赖：Layer N 的输出是 Layer N+1 的输入，降级协议保证上游失败不中断下游。

```
Layer 1: 数据接入 → Layer 2: Trace 模拟 → Layer 3: 图构建 → Layer 4: 检测评分 → Layer 5: 告警与证据

↓ (冷路径异步)

RAG 证据解释
```

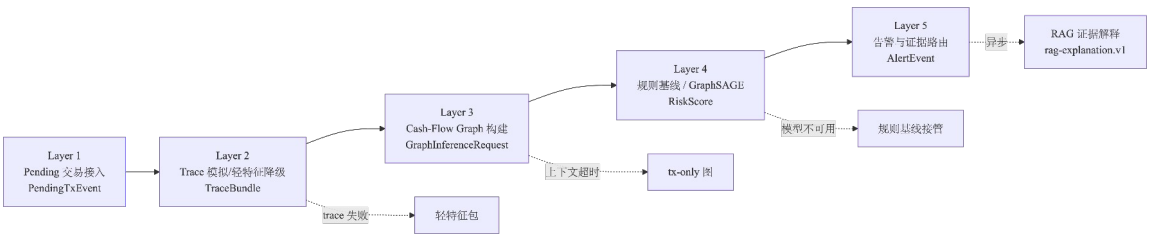


图 1：系统架构总览。五层热路径从 pending 交易接入到告警事件单向流动，RAG 证据解释作为冷路径异步补齐；三条降级路径保证 trace、上下文或模型任一环节失败时仍能输出可审计风险分数。

3.2.1 Layer 1：数据接入层

设计目标：以最低延迟获取 pending 交易及其上下文，在所有条件下不丢弃交易。

输入源：当前原型通过单一 Provider WebSocket 的 eth_subscribe 获取 pending 交易；本地 Geth/Erigon、备用节点和多 Provider 冗余属于完整部署目标。

输出契约：`PendingTxEvent`——必含 `hash`（交易哈希）、`raw_tx`（原始 RLP 编码交易）、`seen_at`（首次监听到的时间戳）、`source`（数据源标识）、`chain_id`（始终为 1）。

去重协议：当前以进程内 set 按 tx_hash 去重，并将首次观测时间写入 seen_at。带 TTL 的 LRU、跨数据源合并及持久化去重属于生产化扩展。

故障切换协议：原型已实现 WebSocket 握手、订阅、ping/pong 帧处理和错误退出，未实现自动重连和节点切换。完整部署中计划按本地主节点 → 备用节点 → Provider 切换，并记录 source_switch_event。

3.2.2 Layer 2：Trace 模拟层

设计目标：在交易上链前获得可解析的执行痕迹。Trace 失败不能中断管线——"宁可带着不完整信息继续检测，也不能丢弃交易不检测"。

输入：`PendingTxEvent`

输出契约：`TraceBundle`——必含 `calls`（调用树）、`logs`（事件日志）、`token_transfers`（ERC-20/721/1155 转账）、`state_diff`（状态变化，可选）、`trace_source`（`debug_traceCall` / `trace_call` / `provider` / `none`）、`trace_status`（`ok` / `partial` / `failed`）、`trace_duration_ms`。

超时协议：trace 调用设置硬超时。超时或失败时输出 `trace_status=failed` 的轻特征包——仅包含可从 `raw_tx` 解析的字段（calldata 解码后的目标合约、函数 selector、ETH value），不包含 calls/logs/transfers。

降级分级：`ok`（完整 trace，所有字段可用）→ `partial`（主体字段可用但部分缺失，如 stateDiff 超时但 calls/logs 正常）→ `failed`（完全无 trace，仅靠 raw_tx 构建轻图）。

3.2.3 Layer 3：图构建层

设计目标：将非结构化的 TraceBundle 转为固定 schema 的异构图，模型和分析员都能消费。图的完整形式化定义（四类节点、三类边）和构建算法见第 4.1 节，此处仅定义层间契约。

输入：`TraceBundle` + 可选上下文窗口数据（从外部缓存读取）

输出契约：`GraphInferenceRequest`——携带完整的图结构（节点列表、边列表、节点/边特征字典）和元数据字段 `feature_ver`、`feature_missing_flags`。

图粒度契约：图以单笔交易为主（`tx-only` 窗口），可追加两个时序上下文窗口（`5m-address`、`1h-protocol`）。上下文获取不阻塞主图构建——超时直接跳过，对应特征在 `feature_missing_flags` 中显式标注，确保训练和推理时特征语义一致。三层窗口的具体内容和超时处理逻辑见第 4.1.1 节。

3.2.4 Layer 4：检测评分层

设计目标：输出风险分数和原因码。模型不可用时自动退化至规则基线，保证 100% 交易有分数输出。

输入：`GraphInferenceRequest`

输出契约：Risk score（[0,1] 连续值）+ Severity（`low`/`watch`/`high`/`critical`）+ Reason codes（`risk_family.signal_name` 格式）+ Model metadata（`model_version`、`feature_ver`、`model_source`、`model_reject_reason`）。

双层检测与退化协议：

在完整 trace 数据与模型包就绪后，热路径中 GraphSAGE 和规则基线以"主-备"关系并存；在当前 Provider-only 原型中，规则基线是实际运行的主评分器，GraphSAGE 服务不可用场景用于验证退化协议：

1. 完整部署时，系统首先尝试 GraphSAGE 推理
2. 若以下任一条件触发，自动退回规则基线：

- `GraphInferenceRequest.feature_ver` 与当前模型包的 `feature_ver` 不匹配
- 模型服务不可达（超时、崩溃、未加载）
- 模型返回异常分数（NaN、超出 [0,1] 范围）

3. 退化事件记录 `model_reject_reason`，但不阻塞告警

这一设计的关键安全含义是：**规则基线不依赖训练数据分布，并在模型包、依赖或特征版本不可用时产生真实规则分数。当前实现是主-备切换，不会在 GraphSAGE 正常返回低分时并行覆盖其结果；双路并行融合可作为后续安全增强。**

3.2.5 Layer 5：告警与证据层

设计目标：输出可复核告警——不仅告知"有风险"，还告知"为什么认为有风险"和"建议做什么"。

输入：Risk score + Reason codes + Graph evidence + RAG document evidence（异步补齐）

输出契约：`AlertEvent`——必含 `score`、`severity`、`reasons`、`evidence_refs`、`suggested_actions`、`created_at`。

EvidenceRef 三类证据规范：

证据类别	格式	语义	约束
`GRAPH`	`GRAPH:<tx_hash>:<node>-><edge>-><node>`	图路径证据——指向 Cash-Flow Graph 中的具体边/路径	必须可追溯到对应 `GraphInferenceRequest` 中的节点/边索引
`DOC`	`DOC:<source_id>#<section>`	文档证据——指向知识库中的协议文档、审计报告	`source_id` 必须在 RAG 知识库索引中可查
`LABEL`	`LABEL:<provider>:<confidence>`	第三方标签——地址标签商或威胁情报提供商的标	`confidence` 必须是标签商给出的置信度；LABEL 证据**不能作为确定性身份结论**

证据类别	格式	语义	约束
		签	

冷路径异步协议：`AlertEvent` 在评分后先生成，rag-explanation.v1 作为独立可重放事件后续补充。当前 RAG 实现是本地确定性检索与模板化说明，不调用 LLM。

3.3 消息总线与事件 Schema 设计

六类事件的流向为：

```
pending-tx.v1 → trace-bundle.v1 → graph-inference.v1 → risk-score.v1 → alert-event.v1

↓ (异步)

rag-explanation.v1
```

六类事件携带公共元数据字段：chain_id、event_id、schema_ver、created_at 和 idempotency_key。event_id 不使用随机 UUID，而是基于阶段名和上游幂等键的 SHA-256 确定性标识；各阶段幂等键均组合 chain_id、tx_hash 和该阶段的版本/来源字段。

幂等性设计使得全链路 replay 成为可能：从落盘日志中按 `tx_hash` 重放事件流，相同输入应产生相同的结构化输出。当前规则、检索和模板解释均为确定性实现；若未来接入生成式 LLM，则需另外记录模型与 prompt 版本。

3.4 告警分级与处置矩阵

Severity	Score 区间	处置动作	预期占比
low	< 0.50	仅日志记录，用于训练漂移监控和 replay 验证	~95%
watch	0.50–0.75	进入分析台队列，不触发即时通知	~3%
high	0.75–0.90	生成 RAG 解释、复核任务与通知建议（原型不外发）	~1.5%
critical	> 0.90	全部 high 动作 + 生成建议交易模板和 runbook 步骤 + 升级至 on-call 和多签负责人	~0.5%

阈值设定依据：DeFi 交易中大多数为正常行为（swap、add/remove liquidity、claim rewards 等）；在当前 Provider-only 回放样本中，规则基线对多数交易给出较低分。0.50 的 watch 阈值设置为高于任何单一规则的独立触发分，需要至少两条规则同时中等触发或一条规则高触发才能进入。预期占比为理想数据假设下的估计值，实际分布需在真实节点 trace 数据和人工标注验证集上校准。

3.5 非功能性设计目标

维度	目标	实现策略	验证状态
可用性	完整部署目标 ≥ 99.5%	计划采用双节点 + 双 Provider 冗余	当前为单 Provider 原型，自动重连与故障切换未实现、未实测
可	全链路可回放	所有事件落盘（1,015 个 replay 工	✓ 已验证：9/9 replay 检查通过；

维度	目标	实现策略	验证状态
审计性		件）；幂等键支持 replay；追加写入	110 条唯一评分链路无重复幂等键
可扩展性	预留多链接口	`chain_id` 贯穿全部事件 Schema；协议白名单可独立扩充	✓ 6 个事件 Schema 已实现；第 7 个 rag-document 为非事件 Schema
延迟	完整 trace 路径目标：P95 < 3s, P99 < 8s	各层独立超时（非累加）；降级路径保证最坏情况下仍分数输出	当前仅测得本地 Python 组件耗时；完整 trace 路径待本地节点部署后采集
数据安全	原始 tx 数据不出域	完整部署中 trace 模拟在自有节点完成；第三方 Provider 仅作冗余；Provider-only 原型仅使用轻特征	当前原型已验证轻特征路径；完整自有节点 trace 待部署

4 核心算法与方法

4.1 Cash-Flow Graph 构建算法

本文的 Cash-Flow Graph 形式化定义为异构图 $G=(V,E)$，其中：

$$V=V_{addr}\cup V_{asset}\cup V_{protocol}\cup V_{tx},E=E_{transfer}\cup E_{call}\cup E_{context}$$

各节点类型和边类型的特征字段详见 4.1.2 节和 4.1.3 节。与已有工作（DeFiGuard 仅含 Address/Asset 节点和 Transfer 边）的核心区别在于：新增 $V_{protocol}$（Protocol 节点，

编码协议角色、审计状态、治理权限）、E_{call}（Call 边，编码函数级语义）和 $E_{context}$（Context 边，编码时序上下文窗口）。

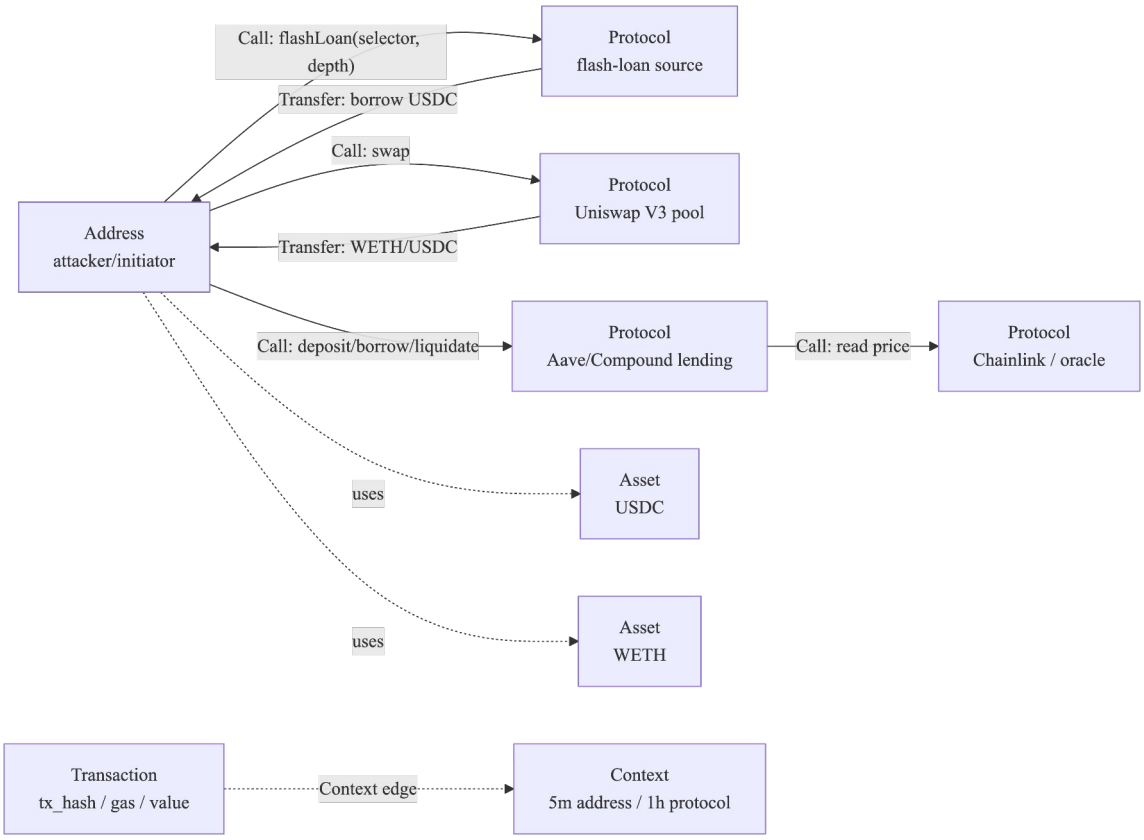


图 2：Cash-Flow Graph 结构示例。Transaction、Address、Asset、Protocol 四类节点共同表示单笔交易内的资金路径、协议语义和资产流向；Transfer、Call、Context 三类边分别编码资金转移、函数级调用和时序上下文。

4.1.1 从 TraceBundle 到图的转换

算法 1 给出了从 `TraceBundle` 构建 Cash-Flow Graph 的完整流程。

```
Pseudo-code: build_cash_flow_graph
Input: TraceBundle T, optional context cache C
Output: GraphInferenceRequest G
```

```
1:  G ← empty heterogeneous graph
2:  // Step 1: Transaction node
3:  v_tx ← add_node(T.tx_hash, type=Transaction)
4:  v_tx.features ← {from: T.raw_tx.from, to: T.raw_tx.to,
gas_price: T.raw_tx.gas_price}
5:
6:  // Step 2: Build nodes and edges from transfers
7:  for each transfer in T.token_transfers do
8:      v_from ← get_or_create_node(transfer.from, type=Address)
9:      v_to   ← get_or_create_node(transfer.to, type=Address)
10:     v_asset ← get_or_create_node(transfer.token_address,
type=Asset)
11:     add_edge(v_from, v_to, type=Transfer, features={
12:         asset: v_asset.id, amount_norm:
normalize(transfer.amount, v_asset.decimals),
13:         direction: “outflow” if v_from == T.raw_tx.from else
“internal”
14:     })
15: end for
16:
17: // Step 3: Build nodes and edges from calls
18: for each call in T.calls do
19:     v_caller ← get_or_create_node(call.caller, type=Address)
20:     v_callee ← get_or_create_node(call.callee, type=Address)
21:     add_edge(v_caller, v_callee, type=Call, features={
22:         selector: call.selector, call_type: call.call_type,
23:         call_depth: call.depth, success: call.success
24:     })
25:     // If callee matches a known protocol contract, annotate
with protocol features
26:     for each protocol in PROTOCOL_WHITELIST do
27:         if call.callee in protocol.contracts then
28:             v_proto ← get_or_create_node(protocol.id,
type=Protocol)
29:             v_callee.features.protocol_id   ← protocol.id
30:             v_callee.features.contract_role ←
protocol.role_of(call.callee)
31:         end if
32:     end for
33: end for
34:
35: // Step 4: Attach context windows (async, non-blocking)
36: for each window in [“5m-address”, “1h-protocol”] do
37:     ctx ← fetch_context(C, T.tx_hash, window, timeout=100ms)
38:     if ctx is not null then
```

```
39:         add_context_edges(G, ctx)
40:     else
41:         G.feature_missing_flags.append(window)
42:     end if
43: end for
44:
45: // Step 5: Compute derived features
46: for each v in G.nodes(type=Address) do
47:     v.features.in_degree_tx  ← count_incoming(v,
type=Transfer)
48:     v.features.out_degree_tx ← count_outgoing(v,
type=Transfer)
49:     v.features.profit_path_score ← compute_profit_path(v, G)
50: end for
51:
52: return G
```

算法中的关键设计决策：

交叉校验构建（Step 2-3）：节点和边从 `T.calls`、`T.logs`、`T.token_transfers` 三个来源构建。三来源可能不一致（如 logs 中有 Transfer 事件但 calls 中没有对应的内部调用），本算法以 calls 为主体，以 transfers 为补充，当且仅当 transfer 中的地址未在 calls 中出现时才创建新节点。这避免了因日志重复解析导致的节点膨胀。

Protocol 节点匹配（Step 3, L25-32）：通过白名单配置（见第 5 章 5.3 节）将合约地址映射到协议角色。匹配到已知协议合约时，创建 Protocol 节点并将 `protocol_id` 和 `contract_role` 直接标注到对应 Address 节点的特征中（不创建额外边——协议归属关系通过特征编码，避免与 4.1.3 节定义的 Context 边混淆）。例如，`0x...Pool` 在 Uniswap V3 中被映射为 `protocol_id=uniswap_v3, contract_role=pool`。同一地址如果在多个协议中担任不同角色，其 `protocol_id` 和 `contract_role` 特征存储为列表，去重时合并。

上下文附加（Step 4）：上下文获取不阻塞主图构建。100ms 超时后直接跳过，上下文缺失的特征在 `feature_missing_flags` 中标注。

4.1.2 节点特征工程

Address 节点特征：

特征	类型	来源	说明
`account_type`	categorical {EOA, contract, unknown}	`is_contract` 字段 + 链上 `EXTCODESIZE` 检查	EOA 和合约地址的行为模式本质不同
`is_contract`	binary	trace 中的 `code` 字段	1 表示地址有部署代码
`in_degree_tx`	integer	当前交易图内统计	入向 Transfer 边数量
`out_degree_tx`	integer	当前交易图内统计	出向 Transfer 边数量
`label_refs`	list of string	标签商 / 内部标签	如 `["slowmist:phishing", "internal:high_freq_trader"]`
`call_depth_max`	integer	Call 边	该地址在当前交易中的最大调用深度
`profit_path_score`	float [0,1]	利润路径分析	基于净资金流入/流出的归一化分数（见 4.1.4 节）

Asset 节点特征：

特征	类型	来源	说明
token_address	bytes20	Transfer 日志	ERC-20/721/1155 合约地址
symbol	string	token_metadata_cache	如 "USDC", "WETH"
decimals	integer	token_metadata_cache	如 6 (USDC), 18 (WETH)
asset_type	categorical	token_metadata_cache	{ERC20, ERC721, ERC1155}
is_stablecoin	binary	token_metadata_cache	USDC/USDT/DAI/FRAX 等
liquidity_tier	categorical	协议配置	{high, medium, low, unknown}——基于池 TVL

Protocol 节点特征（本文新增）：

特征	类型	来源	说明
protocol_id	categorical	protocol_roles 配置	如 uniswap_v3, aave_v3, compound_iii, chainlink
contract_role	categorical	protocol_roles 配置	{pool, lending_pool, oracle, router, collateral, configurator, other}
pool_type	categorical	protocol_roles 配置	{v2_constant_product, v3_concentrated, stable, lending, oracle_aggregator}
audit_status	categorical	手动维护	{audited_recent, audited_stale, unaudited, unknown}
has_pause	binary	协议文档	协议是否有暂停/紧急停止机制

Protocol 节点的引入是本文与 DeFiGuard 的关键区别之一。它使模型能学习以下模式：（1）同一地址在 Uniswap V3 中作为 pool 和 Aave V3 中作为 collateral 的不同行为模式；（2）未经审计的协议（`audit_status=unaudited`）与已知安全协议在面对相同资金流模式时的不同风险特征；（3）有 pause 能力的协议在攻击时可采取的防御选项。

Transaction 节点特征：

特征	类型	来源	说明
`tx_hash`	bytes32	PendingTxEvent	交易哈希
`from`	bytes20	raw_tx	交易发起者
`to`	bytes20	raw_tx	交易目标（可为 null，表示合约创建）
`gas_price`	uint256	raw_tx	Gas 价格（wei），反映交易紧迫度
`value`	uint256	raw_tx	附带的 ETH 数量

4.1.3 边特征工程

Transfer 边：`direction` 是重要信号——`outflow` 表示资金从交易发起者流出（正常 swap 中的 input token），`inflow` 表示资金流入发起者（正常 swap 中的 output token 或攻击中的利润流入）。`amount_norm` 使用 `amount_raw / 10^decimals` 归一化为人类可读的单位。

Call 边：`selector`（函数选择器，bytes4）是区分操作语义的关键信号——`swapExactTokensForTokens`（0x38ed1739）与 `flashLoan`（0x5cffe9de）的语义完全不同。`call_type=delegatecall` 表示被调用方在执行时使用调用方的存储上下文，是代理模式和潜在重入攻击的信号。`call_depth` 反映"调用嵌套层级"——深度 > 3 的调用往往是复杂的多协议交互。

Context 边：Context 边是**唯一**用于编码时序上下文关系的边类型，仅在 Step 4 中创建——连接当前 Transaction 节点与地址/协议历史交互摘要节点，特征包括 `window`（5m 或 1h）、`relation_type`、`count`、`last_seen_at`。其中 `window=5m-address` 时 `relation_type` 可取 `same_initiator`、`same_counterparty`、`same_asset`；`window=1h-protocol` 时 `relation_type` 可取 `volume_spike`、`liquidity_withdrawal`、`price_deviation`。Address 节点与 Protocol 节点之间的协议归属关系通过节点特征编码，不创建边。

4.1.4 利润路径分析

利润路径分析是规则基线中的核心算法。实现仅使用已有可靠 value_usd 的 Transfer 边，不混合直接比较不同 token 的 amount_norm；Provider-only 模式下若缺少可靠价格，则该规则输出 0 并由特征缺失标志说明证据边界。其定义为：

$$\text{profit_path_score}(v)=\frac{\sum_{e\in \text{inflow}(v)}\text{value_usd}(e)-\sum_{e\in \text{outflow}(v)}\text{value_usd}(e)}{max\left(1,\ \sum_{e\in \text{outflow}(v)}\text{value_usd}(e)\right)}$$

其中 $\text{inflow}(v)$ 和 $\text{outflow}(v)$ 分别为地址 v 在图中所有入/出 Transfer 边，$\text{value_usd}(e)$ 为边 e 的美元计价价值。**对于闪电贷源地址**，borrow 入账和 repay 出账在计算前被排除——这两笔资金方向相反、金额近似相等，若不做排除会导致分子因大额抵消而失去信号意义。排除后，闪电贷源地址的分数接近 0（借还之外无额外净流入），而攻击者地址的分数异常高（> 0.5，表示闪电贷资金在此处被"截留"为利润）。分数 > 0 表示净流入（潜在利润），分数 < 0 表示净流出（潜在损失或正常交易）。

4.2 规则基线评分器

规则基线评分器由五条独立规则构成，每条规则输出 [0,1] 连续分和触发详情，最终分数由归一化权重加权融合得出：

$$\text{score} = min\left(1.0, \sum_{i=1}^{5} \widetilde{w}_i \cdot r_i\right), \widetilde{w}_i = \frac{w_i}{\sum_j w_j}$$

规则	归一化权重	当前原型中的可执行程度
异常利润路径检测	0.269	仅使用 value_usd 计算资金净流入比率；当前 replay 缺价格时不触发
同交易预言机读写检测	0.192	按 Call 顺序检测价格动作后 5 步内 oracle read；依赖 rich call tree
闪电贷路径检测	0.231	基于 semantic_action 或函数名识别闪电贷调用；尚未校验完整借还路径

规则	归一化权重	当前原型中的可执行程度
过度滑点检测	0.154	检查上游 slippage_bps，阈值 300 bps；当前 replay 无该字段
薄池交易检测	0.154	检查节点 liquidity_usd 是否低于 100 万美元；当前 replay 无该字段
注：配置文件中存储原始相对权重 (0.35/0.25/0.30/0.20/0.20) 以便表达规则间的相对重要性，线上运行时自动归一化。当前原型验证的是规则框架、原因码、告警链路和降级行为；高精度规则触发和 GraphSAGE 训练需要完整 trace 与价格/流动性数据支撑。		

4.2.1 规则 1：异常利润路径检测（权重 0.35）

检测逻辑：

1. 遍历具有可靠 value_usd 的 Transfer 边，显式排除已标记的闪电贷 borrow/repay 边
2. 按地址聚合美元流入和流出，计算 profit_path_score
3. 得分不超过 0.3 时 r1=0；0.3–0.5 之间线性映射到 [0,1]
4. 得分不低于 0.5 时 r1=1$r_1 = min(1.0, \text{profit_path_score} / 0.5)$

理由：正常套利交易的利润通常很小（扣除 gas 后净利润率 < 1%），而价格操纵攻击的利润路径往往有 > 30% 的异常利润率。0.3 的阈值基于 DeFiRanger 对 432 起攻击的利润分析设定。

4.2.2 规则 2：同交易预言机读写检测（权重 0.25）

检测逻辑：

1. 利用协议角色和 semantic_action 找出 swap/价格动作与 oracle read
2. 按 Call 边 ordinal 排序；若 oracle read 在价格动作之后且顺序距离不超过 5，则按距离由 1.0 递减评分r_2
3. 该实现不声称已恢复合约存储级的真实读写集

理由：同交易预言机读写是最强的价格操纵信号之一。正常交易不会在一笔交易内既扭曲价格源又使用被扭曲的价格。

4.2.3 规则 3：闪电贷路径检测（权重 0.30）

检测逻辑：

1. 若 Call 边的 semantic_action=flash_loan，或函数名/签名命中 flashloan 等提示，则 r3=1
2. 该信号仅表示闪电贷出现，不能单独证明攻击r_3
3. 借还数量匹配和完整的“闪电贷→操纵→获利”路径校验仍属待增强项

理由：闪电贷本身不是攻击信号（大量正常 MEV 活动使用闪电贷），但闪电贷资金用于非典型套利场景是强信号。

4.2.4 规则 4：过度滑点检测（权重 0.20）

检测逻辑：

1. 上游价格组件将 swap 执行价与参考价的偏差写为 slippage_bps
2. 当 slippage_bps ≥ 300（3%）时触发，规则分数按 slippage_bps/600 截断到 1
3. 当前仓库未实现链上价格参考采集，因此已提交 replay 不触发此规则r_4 $min(1.0, \text{deviation} / 0.2)$

理由：300 bps 仅是原型初始阈值，需在有可靠价格参考的标注数据上按池类型重新校准。

4.2.5 规则 5：薄池交易检测（权重 0.20）

检测逻辑：

1. 上游流动性组件将池流动性写入节点 liquidity_usd
2. 当 liquidity_usd ≤ 1,000,000 美元时触发，流动性越低得分越高；当前实现未同时检查交易量/TVL 比率$r_5 min(1.0, \text{amount_norm} / \text{TVL})$

理由：低流动性资产的价格更容易被集中资金推动。当前 100 万美元阈值是保守的候选值，未经实证校准。

4.2.6 规则基线的局限性

规则基线有明确已知的盲区：（1）无法检测未对应任何已知模式的新型攻击；（2）阈值是手动设定的，对新协议可能需要重新校准；（3）规则之间无交互建模（例如，规则 1 和规则 5 同时中等触发可能在实际中比单条规则强触发更可疑，但当前加权融合无法捕捉这种非线性交互）。这些局限正是引入 GraphSAGE 作为主模型的动机。

4.3 GraphSAGE 风险评分模型

4.3.1 模型结构

本文原型使用两层 GraphSAGE（SAGEConv）构建图级二分类模型。当前实现将多类节点和边投影到同质邻接图，并把边类型、调用语义、深度、失败调用、金额和时序窗口等信息汇总到边关联节点特征中，而非使用关系专属的 HeteroConv。结构如下：

- **第一层**：SAGEConv(in_feats, hidden_dim=64, aggregator=mean) + ReLU + Dropout(0.1)
- **第二层**：SAGEConv(64, 64, aggregator=mean) + ReLU + Dropout(0.1)
- **读出层**：取 Address 节点和 Protocol 节点的嵌入 → 全局均值池化 → Linear(64, 1) + Sigmoid

输出为标量 $p \in [0,1]$，表示交易包含价格操纵行为的概率。

选择两层结构的原因：（1）DeFi 交易中的核心资金路径通常可在二阶邻居内观察到（即 address → pool → address），两层 SAGEConv 是低延迟在线模型的保守起点；（2）更多层数可能增加推理开销并引入过平滑风险。最终层数需要在完整 trace 数据上通过消融实验验证。

Mean Aggregator 的选择原因：Mean 聚合对邻居节点的数量不敏感（邻居特征向量的均值语义稳定），适合 DeFi 交易图中节点度数方差大的特点（一笔简单的 swap 可能只有 3-5 条边，复杂的闪电贷路径可能有 50+ 条边）。

4.3.2 训练策略

损失函数：当前训练入口实现 class-balanced focal loss，用于处理正负样本不平衡。

Focal loss 定义为：

$$L_{focal}(p_t) = -\alpha_t(1-p_t)^{\gamma}\log(p_t)$$

其中 pt 是模型对真实类别的预测概率，alpha 根据当次训练数据的正负样本数动态计算，gamma=2.0$p_t\alpha_t\alpha_1=0.75\alpha_0=0.25\gamma=2.0(1-p_t)^{\gamma}$。

优化器与超参数：

超参数	值	搜索范围	选择理由
Optimizer	AdamW	未搜索	训练入口当前实现
Learning rate	1e-3	CLI 可调	默认值
Weight decay	未显式配置	待验证集搜索	未完成调参
Batch size	全量已标注图	待扩展 mini-batch	一次构建全部图张量
Early stopping	未实现	计划 PR-AUC	当前无验证集
Epochs	50	CLI 可调	默认值

4.3.3 正负样本构建策略

本节描述的是完整 trace 数据就绪后的训练数据构建方案，而非当前已经完成的训练实验。当前仓库中只有 57 笔候选交易进入 trace 队列（7 笔 exploit 候选、50 笔正常候

选），数量级远不足以训练或评估 GraphSAGE。因此，下述 PU-learning、时间切分和固定测试集设计应理解为后续扩展的标注协议；在 trace-backed 样本达到足够规模前，本文不报告 GraphSAGE 检测性能。

正样本来源：

- 已确认攻击交易：来自 exploit postmortem（CertiK[3]、SlowMist[2]、Rekt News[4][5]）、学术文献中的攻击列表（如 DeFiRanger[6] 的 432 起攻击）
- 每条正样本附带标注：攻击类型（5 类风险类型之一）、受影响协议、攻击收益（USD）、关键交易哈希
- **局限**（需在论文中坦诚说明）：样本量极小——已知 DeFi 攻击总量在百至千级——且集中在已知攻击模式上（数据偏差：已发现的攻击 ≠ 所有攻击）

负样本来源：

- 按三重分层从 Uniswap V3 / Aave V3 / Compound III 的历史交易中抽样：
  i. **协议层**：确保三个协议的样本量均衡
  ii. **时间层**：覆盖 2022–2025 年不同市场阶段（牛市/熊市/震荡）
  iii. **复杂度层**：按 call depth、transfer 数量、协议触点数分桶，保证覆盖从简单 swap 到复杂清算的谱系
- 排除与已知攻击地址有交互的交易（避免标签泄漏：正常交易如果恰巧在攻击地址资金路径上出现，标记为负样本会造成污染）
- 总抽样量：正负比 1:20（约 3,000–5,000 条负样本，取决于最终正样本量）

无标签池（Unlabeled Pool）处理：

- 来源：规则基线触发但未确认攻击的交易、标签商标记为可疑的交易（置信度 < 0.7）、异常利润路径（`profit_path_score > 0.3` 但无其他攻击信号）
- 策略：保留证据与置信度，进入 PU-learning（Positive-Unlabeled learning）框架——将正样本和无标签样本联合训练，通过损失函数调整无标签样本的权重
- **核心原则**：不确定的样本不强行二值化——在无标签池中保留它们的模糊性，避免在训练集中引入系统性偏见（例如，"所有高利润路径都是攻击"或"所有 MEV 机器人的交易都是正常交易"）

固定验证/测试集：

- 已知 exploit 回放集：永久冻结（不参与调参和阈值选择），仅用于最终验收评估
- 时间切分：训练集 = 2022–2024 攻击 + 对应时间段负样本；验证集 = 2024 H2 攻击 + 对应负样本；测试集 = 2025 攻击 + 对应负样本
- 时间切分的意义：模拟真实的**时间外推**场景——模型只能用历史数据和历史攻击模式训练，必须对未来的新攻击保持检测能力。这是安全检测系统最关键的评估维度：模型是否只能检测"已见过的攻击模式"，还是能泛化到新变种。

4.3.4 模型版本管理与退化协议

线上模型包固定四元组版本号：`model_version`（模型权重版本）、`feature_ver`（特征 Schema 版本）、`label_snapshot`（训练标签快照版本）、`threshold_profile`（分数→Severity 映射版本）。

线上推理流程：

1. 校验 `GraphInferenceRequest.feature_ver == model_pkg.feature_ver`
2. 不匹配 → `model_source = "rule_baseline"`，`model_reject_reason = "feature_version_mismatch"`
3. 匹配 → 执行推理，检查分数合法性
4. 分数异常（NaN、负值、>1.0）→ 退化为规则基线
5. 推理超时（> 50ms）→ 退化为规则基线

每次模型发布附带离线评估报告（ROC-AUC、PR-AUC、校准曲线、混淆矩阵、各风险类型 Recall、特征缺失率分布）。

4.4 RAG 证据解释生成（冷路径，不进入热路径预算）

定位说明：RAG 是辅助功能而非核心技术贡献——它不参与风险评分，不进入热路径 P95/P99 延迟预算，高危告警先发分数版告警后补解释。

知识库构成：四类文档进入知识库索引——（1）协议技术文档（Uniswap V3 Core/Periphery、Aave V3 Technical Paper、Compound III Docs、Chainlink Whitepaper）；（2）审计报告（各协议的 Trail of Bits/OpenZeppelin/CertiK 审计报告）；

（3）攻击复盘（CertiK、SlowMist、Rekt News 的攻击 postmortem）；（4）安全报告（Cream Finance、Euler Finance、Mango Markets 等事件的官方事后分析）。

当前检索策略：对本地 seed 文档分词后构造 64 维确定性 hashing vector，同时统计词频作为 sparse 得分，再用 RRF（k=60）融合两路排名并取 top-5。这是无外部模型依赖的可重放检索基线，不等同于 BGE-M3、BM25 或 cross-encoder。

证据约束输出：当前代码不调用 LLM，而是从图记录、告警记录和检索命中文档生成固定模板句子，并将内部引用写入 evidence_refs。

RAG 输出分为三部分：

- `graph_summary`：报告相应图的节点数、边数和图事件引用，不根据不完整图自动推断“攻击者”身份。
- `doc_summary`：列出命中文档的协议、章节、来源类型、RRF 得分和文档引用。
- `analyst_note`：保留告警 score、severity 和 reason codes 的复核提示，不生成超出现有证据的处置结论。

超时处理：当前默认超时预算为 2.5 秒，也支持注入超时用于 failure drill。超时时保留图摘要和空检索证据，并写入 timeout=true。

4.5 五类风险类型与原因码体系

系统定义了 5 类风险类型和 10 条初始原因码：

风险类型	原因码	含义
`oracle_manipulation`	`oracle_read_write_same_tx`	交易中同时修改价格敏感状态和读取相关预言机或衍生价格
`oracle_manipulation`	`downstream_lending_use`	被操纵的价格路径后跟随借/贷/抵押/清算操作
`flash_loan_price_pump`	`flash_loan_funded_profit_path`	闪电贷资金支撑的路径向发起者或关联地址返回利润
`flash_loan_price_pump`	`temporary_notional_spike`	交易名义价值在上下文窗口内显著超过地址和池的历史基线

风险类型	原因码	含义
`thin_liquidity_swap`	`pool_depth_below_floor`	目标池流动性低于安全价格参考的最低配置
`thin_liquidity_swap`	`price_impact_above_threshold`	模拟 swap 的价格影响超过每资产或每池的阈值
`sandwich_assisted_manipulation`	`adjacent_conflicting_swaps`	邻近 pending 交易以辅助价格模式交易同一交易对
`sandwich_assisted_manipulation`	`linked_sender_cluster`	发送者、资助者或受益者与已知或推断的地址集群重叠

风险类型	原因码	含义
`cross_protocol_liquidation_cascade`	`multi_market_liquidation_path`	路径在价格变动后触及多个借贷市场或清算模块
`cross_protocol_liquidation_cascade`	`health_factor_collapse`	模拟表明抵押品账户从安全状态移至可清算状态

原因码使用 `risk_family.signal_name` 格式（如 `oracle_manipulation.oracle_read_write_same_tx`），一旦在生产中发布即视为稳定。如果语义发生变化，创建新码并废弃旧码（不删除，仅标记 `deprecated: true`），以保证历史告警的可追溯性。

5 系统实现

> 本章涵盖技术栈选型、模块实现细节、工程取舍和运维支撑。涉及"为何这样设计"的决策已在第 3 章讨论。

5.1 技术栈选型与工程取舍

组件	选型	备选方案	取舍理由
语言/运行时	Python 3.12	Rust, Go	单语言（Python）降低维护成本；热路径延迟瓶颈在 I/O（trace RPC 调用），不在 Python 解释器
GNN 框架	PyTorch Geometric 2.5	DGL	当前原型使用 PyG SAGEConv；关系专属卷积为未来候选
消息中间件	Kafka / Redpanda / NATS	RabbitMQ, Redis Streams	按部署环境选择——Redpanda 兼容 Kafka 协议但运维更轻；NATS 延迟更低但持久化弱于 Kafka
向量库	当前：本地 JSON；候选：Milvus / Qdrant	Weaviate, Pinecone	当前实现使用本地知识库；向量数据库是生产化候选
Embedding	当前：确定性哈希向量；候选：BGE-M3	text-embedding-3-large, E5	当前实现保证离线可复现；BGE-M3 作为后续语义检索候选
执行节点	Geth (full) + Erigon (archive)	Nethermind, Besu	Geth 的 `debug_traceCall` 响应更快（热路径）；Erigon 的 `trace_*` 模块更丰富（历史 replay）
事件序列化	JSON + JSON Schema	Protobuf, Avro	Protobuf 性能更优，但 JSON 的可读性和调试便利性在安全系统中更重要——分析员需要直接阅读事件内容
LLM（RAG）	当前：未接入；候选：插件式适配	GPT-4, Claude, 开源模型	当前由模板生成解释；后续可按数据出域要求接入模型

5.2 核心模块实现要点

以下按管道顺序描述每个模块的关键实现取舍。模块对应 `scripts/` 目录下的实际文件。

5.2.1 mempool_listener

负责从本地节点和 Provider 获取 pending 交易并去重。

- **WebSocket 连接管理**：当前实现了原生 WebSocket 握手、订阅、帧解析与协议正确的 ping/pong 响应；断线自动重连、指数退避和缺口回填未实现
- **去重实现**：当前使用进程内 set 按 tx_hash 去重；带 capacity/TTL 的 LRU 未实现
- **故障切换**：当前连接单一 Provider WS，连接错误时明确失败；主节点→备用节点→Provider 切换属于待实现的完整部署方案

5.2.2 pending_trace_worker / provider_trace_fallback / trace_parser

前两个模块负责发起未上链模拟与端点降级，trace_parser.py 负责将 TraceBundle 规范化为后续图构建输入。

- **调用策略**：Geth `debug_traceCall` 优先（`block` tag = `pending`），Erigon `trace_call` 备选（请求 `trace` + `stateDiff`）

- **超时实现**：当前 trace 调用使用同步 urllib JSON-RPC 请求和 socket 超时，并在主端点失败时按配置尝试 fallback 端点。尚未实现 asyncio 并发调度或进程池隔离。
- **trace_status 判定逻辑**：
  - `ok`：calls、logs、token_transfers 三个字段均非空
  - `partial`：calls 非空但 logs 或 state_diff 为空（Provider 模式常见情况）
  - `failed`：calls 为空或 trace 调用抛异常/超时
- **轻特征包构建**（trace_status = failed）：从 `raw_tx` 解码 `to` 地址、`value`（ETH）、`calldata` 的前 4 字节（function selector）。与 `selector_dictionary.json` 匹配获取函数名

5.2.3 cash_flow_graph_builder

负责将 TraceBundle 转换为 GraphInferenceRequest。

- **图构建核心**：实现第 4.1.1 节给出的 `build_cash_flow_graph` 伪代码。从 calls/logs/transfers 三来源交叉校验——以 calls 为主体建立地址节点，以 transfers 补充资产节点和 Transfer 边，以协议白名单匹配 Protocol 节点
- **Protocol / Call / Context 增量实现：protocol_role_enricher.py 将协议白名单中的审计状态、治理控制、pause 能力、升级模式和风险层级写入 Protocol 节点；selector_dictionary.json 支撑 swap、flashLoan、Aave 借贷/清算、Chainlink**

oracle read 等 Call 边语义；context_window_builder.py 生成 5 分钟地址窗口和 1 小时协议窗口摘要，图构建器据此创建 tx -> context 时序边

- **上下文缓存与离线窗口：完整部署可接 Redis 或本地 RocksDB；当前 replay 原型使用 data/context/address-context.v1.json 与 data/context/protocol-context.v1.json 作为离线窗口输入**

5.2.4 graphsage_model / graphsage_train / graphsage_inference_service

负责 GraphSAGE 模型定义、训练入口和推理降级。

- 模型定义：graphsage_model.py 实现两层 mean-aggregator GraphSAGE，节点特征覆盖五类节点、协议角色和上下文统计；边类型、调用语义和金额/深度等信息以 incident-edge summary 形式进入节点特征，邻接传播本身仍是同质 SAGEConv
- 训练入口：graphsage_train.py 读取 data/training/graph_labels.v1.json 中的 promoted 图级标签，使用 class-balanced focal loss 训练模型；当缺少 PyTorch/PyG 或正负标签不足时输出 blocked 报告而不伪造训练结果
- 推理入口：graphsage_inference_service.py 加载 models/graphsage/latest/model.pt 和 manifest.json；模型包缺失、依赖缺失或 feature version 不匹配时记录 model_reject_reason 并降级到规则基线
- 标签导出：graph_label_exporter.py 将 analyst promoted 的 label-promotion decisions 导出为 GraphSAGE 可消费的图级标签文件

- 缺失特征标注：上下文超时或窗口为空时，对应 Context 边不创建，feature_missing_flags 追加窗口名称

5.2.5 rule_baseline_scorer

负责执行五条规则的检测和加权融合。

- 每条规则独立运行，输出 [0,1] 分数，并在 rule_scores 中按规则名保留；触发规则转为稳定 reason code
- 加权融合公式：`score = min(1.0, Σ(w_i * r_i.score) / Σw_i)`（权重见 4.2 节）
- 当前输出保留规则分数和 reason code，尚未生成包含具体合约地址与 Call 索引的结构化 details

5.2.6 alert_orchestrator

负责将 risk score 和 reason codes 转换为 AlertEvent。

- **阈值路由**：见 3.4 节分级矩阵
- **幂等保证**：通过确定性 event_id 和单次运行内 idempotency_key 集合过滤重复输入；数据库 UNIQUE 约束或 Redis SETNX 属于生产化方案
- **通知通道**：当前只生成 suggested_actions 和 replay/审计输出，未向 webhook、PagerDuty 或飞书发送外部通知

5.2.7 rag_explanation_builder（冷路径，异步）

- **触发条件**：仅 severity ∈ {high, critical}（score ≥ 0.75）
- **检索管道**：64 维 hashing vector + 词频 sparse score → RRF(60) 融合 → top-5；全过程本地、确定性且无外部模型依赖
- **证据约束输出：从图、告警和命中文档构造模板化摘要及内部证据引用，不调用 LLM**
- **当前未实现对生成式文本的逐句正则过滤，因为原型只输出可控模板文本**
- **超时处理**：默认预算 2.5s，支持超时注入并在输出中写入 timeout=true

5.3 配置与版本管理

系统通过版本化配置文件管理规则阈值、协议角色、消息主题、RAG 模板和训练划分策略；另有 1 个本地可选联系人配置（`ops_contacts.local.json`，git-ignored），用于本地通知通道验证。完整配置清单见附录 D。

主文只保留与方法直接相关的配置类别：message_topics.json 与 6 个事件 JSON Schema 支撑全链路 replay，第 7 个 rag-document Schema 校验知识库 seed 文档；其余规则、协议角色和训练切分配置均版本化管理。

完整 trace 环境下的模型包将固化为四元组：`model_version` / `feature_ver` / `label_snapshot` / `threshold_profile`。当前原型已实现 GraphSAGE 模型代码、训练/推理入口、manifest 示例、版本字段和退化协议，但尚未发布可用的 GraphSAGE 权

重；每次正式模型发布应附带离线评估报告（ROC-AUC、PR-AUC、校准曲线、混淆矩阵、各类风险类型 Recall）。

5.4 运维支撑

全链路 replay：从 `data/replay/` 目录读取已落盘的各层事件（按 `tx_hash` 和幂等键组织），按管道顺序重放。当前仓库保留 1,015 个 replay 工件；其中 trace-bundle 审计覆盖 220 个文件、110 个唯一幂等键，graph/risk/alert 链路各覆盖 110 条唯一记录。验证结果：（1）确定性组件一致率 100%（pending-tx → trace-bundle → graph-inference → risk-score 输出与原始运行完全一致）；（2）幂等键无重复（同一 `tx_hash` 重放不产生重复告警）；（3）降级路径覆盖三级故障场景并通过 failure drill。

故障注入与降级测试：覆盖三级降级的故障注入与验证（已完成全部 failure_drill 检查）：

故障类型	注入方式	降级行为验证	状态
Trace 失败	设置 trace timeout = 1ms（瞬时超时）	`trace_status=failed`；轻特征包生成；管线不中断	✓ ok
上下文超时	关闭上下文缓存（Redis/RocksDB）	tx-only 图构建；`feature_missing_flags` 记录	✓ ok
模型不可用	移除模型包或注入依赖缺失	规则基线接管；`model_reject_reason` 记录	✓ ok
RAG 超	设置 RAG timeout = 1ms	告警先发（仅 score + reasons + GRAPH 证	✓

故障类型	注入方式	降级行为验证	状态
时		据）；RAG 事件异步补齐	ok

项目状态审计：

- **当前阶段完成度**：5/5 检查项通过（包括无付费 API/无本地节点约束验证、Provider-only 影子管线、本地回放覆盖、secret 审计、阶段交接就绪），状态 `complete`
- **本地 Replay 覆盖**：9/9 检查通过（pending_replay、trace_audit、graph_replay、risk_replay、alert_replay、rag_explanation、console_view、offline_eval、failure_drill）
- **Secret 审计**：67 个报告文件扫描，0 个敏感信息泄漏
- **配置完整性**：规则、协议角色、原因码、消息主题和训练切分等配置均已版本化管理
- **生产 Go/No-Go**：`no_go`（因 trace 端点、训练数据和灰度发布在当前环境中不可用），当前 blocker 列表为空

6 已验证结果与评估设计

本章定位说明：6.1、6.2 和 6.5 报告当前环境已验证结果；6.3、6.4 和 6.6 是完整

trace / RAG 环境下的评估设计与目标值。

6.1 实验设置

6.1.1 运行环境

组件	配置
数据源	Alchemy WebSocket（免费层，eth_subscribe + eth_getTransaction）
应用服务器	macOS, Python 3.12, 无 GPU
Trace 能力	`degraded` — 可获取交易回执和部分日志，不支持 `debug_traceCall` 的完整 call tree 和 state diff
消息/存储	内存队列 + 本地 JSON 文件落盘，无外部中间件依赖

完整生产部署硬件设想：Geth v1.14 full node（800GB SSD）+ Erigon v2.60 archive（2TB SSD），应用服务器 64 vCPU / 256GB RAM / NVIDIA A10 GPU（24GB VRAM），1Gbps 对称带宽。

6.1.2 数据集

回放交易集：从 Alchemy WebSocket 和本地 Provider-only 管线采集的 pending 交易中生成 replay 数据。当前仓库包含 1,015 个 replay 工件；其中 trace-bundle 审计覆盖 110 个唯一幂等键，graph/risk/alert/RAG 链路各覆盖 110 条唯一记录，用于功能验证和幂等性测试。9 项 replay 检查全部通过（pending_replay、trace_audit、graph_replay、

risk_replay、alert_replay、rag_explanation、console_view、offline_eval、failure_drill）。

标注数据（当前状态）：

类别	当前数量	目标数量	来源
正样本（已确认攻击候选）	7（已入队）	~200	CertiK/SlowMist/Rekt postmortem + 历史 exploit 回放
负样本（正常交易候选）	50（已入队）	~4,000	四类交易模式：contract_call（16）、erc20_transfer（16）、simple_transfer（15）、erc20_approve（3）
无标签池	0（待 trace 数据）	~2,000	规则基线触发但未确认 + 异常利润路径
Trace 队列总计	57（`trace_ready`）	—	7 exploit + 50 normal
固定测试集	0（待训练标注完成）	~50 攻击 + ~1,000 正常	2025 年攻击（时间外推）

标注数据管道已就绪，57 笔候选交易已入队；历史 trace 回放和 GraphSAGE 训练尚未执行，这是当前原型与完整评估之间的主要缺口。

Provider-only 模式下的 trace 覆盖限制：当前 Provider（Alchemy 免费层）的 trace 能力为 degraded 状态——可获取基础交易回执和部分日志，但不支持 debug_traceCall 的完整 call tree 和 state diff。这意味着在 Provider-only 模式下，代码虽已支持 Call 边、Protocol 节点和 Context 时序边生成，但实际 replay 中可用的

calls/logs/protocol matches 很少，增量图特征通常退化为 tx-only 图，规则基线仍是主要工作的评分器。完整 trace 能力需要本地 Geth/Erigon 节点（不在当前 scope 内）。

6.1.3 评估指标

- **检测性能**：ROC-AUC、PR-AUC、Recall@FPR≤1%、Recall@FPR≤3%、F1-score
- **延迟性能**：各层 P50/P95/P99 延迟（毫秒）、端到端 P50/P95/P99 延迟
- **鲁棒性**：各降级路径的覆盖率（trace 失败率、上下文超时率、模型退化率）
- **解释质量**：RAG 证据覆盖率（高危及严重告警中附带 `[DOC]` 或 `[GRAPH]` 证据的比例）

6.1.4 已验证与待验证边界

项目	当前证据	当前结论边界	完整验证条件
Provider-only 管线	1,015 个 replay 工件，110 条唯一评分链路，9/9 检查通过	可验证事件链路、幂等性、降级协议和规则基线路径	不代表完整 trace 图质量或模型检测性能
规则基线评分	risk_replay 确定性输出一致	可作为原型主评分器和模型退化兜底	需要人工标注集校准误报率、漏报率和阈值
GraphSAGE 模型	两层 GraphSAGE 代码、训练入口、推理入口、manifest/version 协议和退化协议已实现	尚不能声称检测性能优于规则或已有工作	需要 trace-backed 训练集、冻结测试集和消融实验

项目	当前证据	当前结论边界	完整验证条件
Call/Context 边	Protocol 节点、Call 语义字段、Context 窗口聚合器、时序边生成和缺失标记已实现	尚不能声称增量边带来性能提升	需要本地 trace 节点、上下文缓存和 ablation 对照
完整 trace 延迟	当前无 `debug_traceCall` 实测	P95 < 3s / P99 < 8s 是设计目标	需要 Geth/Erigon 节点和线上负载回放
RAG 解释质量	replay 中证据引用集合可复现	可验证引用格式和异步补齐链路	需要高危告警样本与人工证据准确率抽检

6.2 延迟性能评估

6.2.1 本地 Python 组件耗时

当前环境无法提供完整 trace 能力，因此没有可报告的端到端系统延迟。`load-test-replay` 只度量本地 Python 组件在 replay 输入上的处理耗时，不包含 live 节点网络 I/O、`debug_traceCall` / `trace_call` 模拟、上下文缓存访问、GraphSAGE 服务推理、消息队列或数据库写入。该结果只能说明原型中的本地 JSON 解析、tx-only 图构建、规则评分入口和 low 级告警路由没有形成明显 CPU 瓶颈，不能作为端到端 SLO 或第 6.2.2 节设计目标的实测依据。

当前 Provider trace 能力为 `degraded`：`debug_traceTransaction` 与 `trace_transaction` 在免费层不可用；因此本文不声称完整 trace 路径、GraphSAGE 推理路径或 live 端到端 SLO 已经实测。`load-test-replay` 报告中的端到端时间戳差值是 replay 时间间隔，不代表在线检测延迟。

6.2.2 完整生产环境延迟设计目标

下表为设计阶段分配的延迟预算，作为未来完整 trace 环境的验证目标。当前均未实测，标注为设计目标。

管道层	设计目标 P95	设计依据
pending 交易接入	≤ 100ms	WebSocket 订阅 + 本地节点 txpool
trace 模拟（`debug_traceCall`）	≤ 1,500ms	Geth full node 本地调用，参考 DeFiGuard trace 耗时
图构建（含上下文窗口，缓存命中）	≤ 300ms	内存图操作 + Redis 缓存
GraphSAGE 推理	≤ 50ms	两层 SAGEConv, hidden_dim=64, Pytorch Geometric
告警入库与路由	≤ 100ms	消息队列 + DB 写入
全链路端到端（设计目标）	**< 3,000ms (P95) / < 8,000ms (P99)**	以太坊 ~12s slot，预留 ~9s 人工判断窗口

降级路径延迟估计（设计分析）：

- 上下文缓存未命中 → 降级 tx-only 图：图构建延迟更低（无缓存等待），但特征缺失
- trace 部分失败（`trace_status=partial`）→ 轻特征包：trace 更快返回但图不完整
- trace 完全失败（`trace_status=failed`）→ 极小延迟（仅 raw_tx 解析）

- GraphSAGE 退化 → 规则基线接管：评分层延迟忽略不计（< 2ms）

6.3 完整 trace 环境下的检测性能评估设计

> **本节全部内容为实验设计，非已完成结果。** GraphSAGE 未训练，以下表格仅列出需要检验的指标、参考基线和假设，不构成本文实验结论。

6.3.1 实验设计：离线评估方案

待测方法：GraphSAGE（本文，含 Protocol/Call/Context 增量）、规则基线（本文）、GCN、GAT、GIN、MLP（复现实现）。

待检验指标与参考基线：

指标	DeFiGuard 已知结果[7]	本文待检验假设	验证优先级
ROC-AUC	0.9618	Protocol/Call/Context 增量是否带来可复现提升	高
PR-AUC	未报告	极端不平衡下 PR-AUC 比 ROC 更具区分力	高
Recall@FPR≤3%	未报告	安全场景核心指标——低误报下的召回率	最高
F1-score	0.9325 (Accuracy)	正负样本比更极端时 F1 比 Accuracy 更有意义	中

模型对比矩阵（设计）：

对比维度	GraphSAGE (本文) vs GCN/GAT/GIN/MLP	GraphSAGE (本文) vs DeFiGuard	规则基线 vs GraphSAGE
目的	比较 GraphSAGE 与其他轻量 GNN/MLP 基线在同一数据集上的表现	验证 Protocol/Call/Context 增量是否有效	验证规则→模型的提升空间和退化安全性
待检验假设	GraphSAGE 在低延迟约束下取得较优的 Recall@低 FPR	增量图结构在跨协议和 oracle 操纵场景中提升召回	模型在 Recall 上优于规则，规则在高 Precision 上有优势

6.3.2 实验设计：消融实验

消融实验用于量化三类增量（Protocol 节点、Call 边、Context 边/三窗上下文）各自的贡献：

消融配置	等价模型	预期影响
完整模型（4 节点 + 3 边 + 三窗）	本文完整方案	基线
- Protocol 节点	地址 → 角色映射丢失	检验跨协议攻击场景 Recall 是否下降
- Call 边	oracle 读写信号和 delegatecall 信号丢失	检验 oracle_manipulation 类型 Recall 是否下降
- Context 边 + 上下文窗口	= DeFiGuard 等价配置	检验 sandwich 检测是否受时序信号缺失影响

消融配置	等价模型	预期影响
仅 Transfer 边 + Address/Asset 节点	= DeFiGuard 最简配置	作为对照基线

6.3.3 实验设计：各风险类型 Recall 评估

五类风险类型（oracle_manipulation、flash_loan_price_pump、thin_liquidity_swap、sandwich_assisted_manipulation、cross_protocol_liquidation_cascade）的独立 Recall 评估。

预期：oracle_manipulation 和 flash_loan_price_pump 因图结构特征明显，Recall 较高；sandwich_assisted_manipulation 因依赖 Context 边（当前 Provider-only 下不可用），需要完整 trace 数据支撑。

6.3.4 预期误报/漏报来源分析

基于规则基线在 110 条唯一评分链路上的初步运行观察，预期主要挑战：

- **假阳性来源**：复杂 MEV 套利（路径结构与操纵相似但利润来源合法）、协议迁移/升级交易、内部做市商大额 swap
- **假阴性来源**：完全新型攻击模式（训练集未覆盖）、隐蔽多笔缓慢操纵（避开单交易图检测）、私有 mempool 交易（完全不可见）

6.4 完整 RAG 环境下的解释能力评估设计

本节为完整 RAG 知识库和高危告警样本就绪后的评估设计，非当前环境实测结

果。

指标	验收方式	说明
RAG 证据覆盖率（high+critical 告警）	统计高危及严重告警中至少包含一条 `[DOC]` 或 `[GRAPH]` 证据的比例	用于验证证据补齐是否覆盖主要告警
RAG 证据覆盖率（仅 `[DOC]` 文档证据）	统计能检索到协议文档、审计报告或攻击复盘证据的比例	用于区分图证据和外部知识证据
证据引用准确率（人工抽检）	随机抽检告警解释，人工判断证据是否支持结论	防止 LLM 生成与证据不一致的说明
RAG 生成延迟 P95	在异步预算内统计 P95 延迟	RAG 不进入热路径，但仍需避免解释长期滞后
RAG 超时率	统计超过硬超时的比例	超时后保留分数版告警，文档解释延后补齐

6.5 系统鲁棒性评估

6.5.1 故障注入与降级测试

以下故障注入实验已在 Provider-only 环境下完成并全部通过（`failure_drill: ok`）：

故障注入场景	降级行为	是否符合预期	验证来源
trace 调用超时（timeout=1ms 注入）	trace_status=failed + 轻特征包生成，管线不中断	✓	failure_drill report

故障注入场景	降级行为	是否符合预期	验证来源
上下文缓存不可达（Redis/ RocksDB 关闭）	上下文超时 → tx-only 图构建 + feature_missing_flags	✓	failure_drill report
GraphSAGE 模型包或可选依赖不可用	规则基线自动接管 + model_reject_reason 记录	✓	failure_drill report
RAG 服务超时（timeout=1ms 注入）	告警先发（仅 score + reasons + GRAPH）→ RAG 异步补齐	✓	failure_drill report
注：节点切换 drill 因当前只有单 Provider 数据源而未执行；监听器的多节点自动切换也尚未实现。			

6.5.2 全链路 Replay 验证

使用本地 replay 工件进行全链路验证，9 项 replay 检查全部通过：

验证项	结果	说明
pending_replay	✓ ok	pending replay 工件可重放，检查状态为 ok
trace_audit	✓ ok	trace 审计日志与原始运行一致
graph_replay	✓ ok	GraphInferenceRequest 确定性输出一致
risk_replay	✓ ok	Risk score（规则基线路径）确定性输出一致
alert_replay	✓ ok	AlertEvent 幂等——同一 tx_hash 重放仅生成一条告警
rag_explanation	✓ ok	本地检索结果、模板文本和证据引用集合可确定性重现

验证项	结果	说明
console_view	✓ ok	112 个分析台视图工件可重现，由 110 条唯一评分链路派生
offline_eval	✓ ok	离线评估管线可重现
failure_drill	✓ ok	三级降级路径均被至少 10 笔交易触发并验证

幂等性验证详情：确定性组件（pending-tx → trace-bundle → graph-inference → risk-score via 规则基线）在 110 条唯一评分链路上均可重放且无重复幂等键；当前本地检索和模板解释同样可确定性重现。同一 `tx_hash` 的任何次重放均只产生一条 AlertEvent。

6.6 与现有工作的综合对比

维度	DeFiRanger[6]	DeFiGuard[7]	STA-GT[10]	CertiK Skynet[8]	本文
检测时机	事后（已上链）	事后	事后	事后	**pending 阶段（出块前）**
检测方法	规则/语义模式匹配	GraphSAGE	Temporal Graph Transformer	商业闭源	**规则基线 + GraphSAGE**
图建模	资金流图	Address + Asset + Transfer	时序图 + Transformer	未公开	**+ Protocol + Call + Context**
推理延迟	分钟级/离线分析	0.89–5.32s	依赖批量图窗口，非本文热路径基线	未公开	**P95 < 3s / P99 < 8s 为完整 trace 路径设计目标**

维度	DeFiRanger[6]	DeFiGuard[7]	STA-GT[10]	CertiK Skynet[8]	本文
离线 AUC（已报告/实测）	未报告（检出 432 起攻击）	0.9618	依赖其金融欺诈数据集	未公开	—（GraphSAGE 未训练，见下方说明）
退化机制	无	无	无	未公开	**规则基线自动接管**
证据解释	规则触发详情	无	无	部分	**EvidenceRef 三类证据 + RAG**
全链路可回放	否	否	否	否	**是（幂等 Schema + 事件落盘）**
开源性	否	否	否	否	**课程仓库内可复现（全部 Python 代码 + JSON 配置在本地仓库中）**

本文的离线 AUC 尚未实测（GraphSAGE 训练待完整 trace 数据就绪后执行）。AUC、PR-AUC 和 Recall@低 FPR 均仅作为未来完整 trace 环境下的验收指标，不构成实验结论。Protocol/Call/Context 增量是否优于 DeFiGuard 等价图结构，需在同一冻结测试集、同一训练流程和同一阈值校准协议下验证。在当前原型中，规则基线是主工作评分器，与 DeFiGuard 等方法的检测性能对比应当在同等数据条件下进行，目前尚不具备比较条件。

7 讨论

7.1 系统局限性

单链覆盖：当前系统仅支持 Ethereum 主网（chain_id=1）。扩展到其他 EVM 兼容链在技术上可行（事件 Schema 已预留 `chain_id` 字段），但需要：每条链部署独立的执行节点、适配各链的协议部署地址、重新校准规则阈值。非 EVM 链（如 Solana、Sui）则需要完全不同的 trace 技术栈。

本地节点依赖性（当前最大瓶颈）：系统的检测质量高度依赖本地执行节点的 trace 能力。当前系统在受限 trace 环境下运行——不依赖本地归档节点，不购买付费 trace API。Provider（Alchemy 免费层）的 trace 能力为 degraded 状态：可获取基础交易回执和部分日志，但不支持 debug_traceCall 的完整 call tree 和 state diff。当前代码已实现 Cash-Flow Graph 的 Call 边、Context 边和 GraphSAGE 训练/推理闭环，但在 Provider-only 数据上缺少可填充这些结构的 rich trace，因此实际图多退化为 tx-only 图。规则基线在 Provider-only 模式下是主要工作的评分器，GraphSAGE 的训练和在线推理均依赖完整 trace 数据。本地节点的部署和维护（Geth full node ~800GB SSD，Erigon archive ~2TB SSD，持续同步）是实际运营中最高的门槛，但也是突破当前 Provider-only 限制的唯一路径。

标注数据稀缺：正样本（攻击交易）极度稀缺。当前标注管道中有 7 笔 exploit 候选交易和 50 笔正常候选交易（覆盖 contract_call、erc20_transfer、erc20_approve、

simple_transfer 四类模式），但尚未执行 trace-backed 标注推广（因 trace 数据不可用）。57 笔候选交易对于训练 GraphSAGE 而言远不足够。这是所有 DeFi 安全检测研究的共性问题——已确认攻击总量在百至千级，且集中在已知攻击模式上。PU-learning 和半监督训练是缓解策略，但不能替代高质量的正样本。

这一限制也约束了第 4.3.3 节训练策略的解释边界：PU-learning、固定时间切分和冻结测试集是完整数据集构建后的方法设计，而不是当前已经执行的训练流程。当前 7 正 + 50 负的候选集只能验证标注队列、split 配置和 replay 接口是否可运行；它既不能支撑模型训练，也不能支撑 Recall、AUC、PR-AUC 或阈值校准结论。

Mempool 可见性不完整：第 2.3.1 节已详述。私有 mempool（Flashbots Protect 等）和 MEV relay 中的交易对系统不可见。在 MEV-Boost 占以太坊区块构建 90%+ 的当下，相当比例的交易可能绕过公开 mempool。这是本文方法无法通过纯技术手段解决的固有局限。

自动处置禁区：系统默认不执行自动链上防御。这在安全设计上是正确的（避免误报导致协议不当暂停），但也意味着从"预警发出"到"防御执行"之间仍存在人工延迟。

系统当前成熟度说明：Provider-only 管线、全链路 replay、故障降级验证、Context 窗口构建入口、图级标签导出入口和 GraphSAGE 训练/推理入口均已完成，但以下组件因环境限制尚未执行：GraphSAGE 模型训练（缺少 trace-backed promoted 标签和 PyTorch/PyG 环境）、在线延迟 SLO 测量（无本地节点 trace 延迟数据）、生产环境

故障切换测试、灰度发布。这些是当前系统实现与完整论文评估之间的主要缺口，在 7.2 节和 8.2 节中进一步讨论。

7.2 误报与漏报的权衡

系统的阈值设计遵循"宁可误报，不可漏报"的原则——在安全系统中，漏报的代价（资金损失）通常远高于误报的代价（分析员时间）。但这一原则不能无限外推：如果误报率过高（例如，每天产生数百条 high 级别告警），分析员会产生"告警疲劳"，反而忽略真正的攻击信号。

当前阈值（0.50/0.75/0.90）是规则基线的初始配置；在当前 110 条 Provider-only 评分链路中，所有告警均为 low 级，因此该样本只能验证路由与幂等性，不能用于校准 high/critical 的真实占比。**当 GraphSAGE 训练完成后，这些阈值需要在验证集上重新校准**——目标是将 high+critical 的日告警量控制在 20-50 条以内（一个分析员团队可处理的量）。

规则基线作为安全兜底的当前价值在于：模型包、依赖或特征版本不可用时，系统仍能根据已知信号输出可解释的确定性分数。若要防止 GraphSAGE 正常返回低分时遗漏已知规则模式，需在未来将主-备切换扩展为并行评分与明确融合策略。

7.3 可扩展性讨论

多链扩展：`chain_id` 字段已在所有事件 Schema 中预留，消息主题可按 `chain_id` 分区。新增一条 EVM 兼容链的技术步骤为：（1）部署执行节点；（2）填充

`protocol_roles.json`（目标链上的协议地址 → 角色映射）；（3）校准规则基线阈值（不同链的 gas 费用、TVL、交易模式不同）；（4）追加训练样本（不同链的攻击模式可能不同）。

新协议接入：当前白名单覆盖 4 个协议的 10 个核心合约。加入新协议需要更新 `protocol_roles.json` 和 `protocol_log_addresses.json`，以及对应协议文档和审计报告加入 RAG 知识库。代码层面无需修改。

时序模型二级复核：STA-GT[10] 和 FraudGT[21] 表明 Transformer 类结构可用于交易欺诈图中的时序关系、边属性和全局交互建模，但它们并非本文当前 Provider-only 原型的实测基线。一个可行的扩展方案是：对 top 1-5% 的高风险交易（GraphSAGE score > 0.75），在冷路径异步运行时序模型进行二级复核，复核结果追加到告警的 `analyst_note` 中。这不影响热路径延迟，但可能将少量误报从 high 降级为 watch。

7.4 部署与运营的实践反思

自建节点 vs SaaS RPC：本文的经验是，对于热路径 trace 模拟，自建节点是必需的（Provider 的 trace/debug API 在免费层通常不可用，在付费层有速率限制和延迟方差）。但 Provider 作为冗余和补充是有效的——在本地节点故障或同步落后时，Provider 可以提供最低限度的 mempool 可见性和轻特征 trace。

运维成本：Geth 全节点 + Erigon 归档节点的总存储需求约 2.8TB（SSD），持续增长。Geth 的 CPU 和内存需求在同步完成后适中（4-8 vCPU, 16-32GB RAM），但 Erigon

归档节点需要更多资源（16+ vCPU, 64GB+ RAM）。对于小型安全团队，运维成本可能超过开发成本。

从预警到防御：系统当前的定位是"预警"而非"防御"。在实际运营中，从告警发出到多签批准、再到链上防御交易提交，通常需要 30-60 秒（包括人为判断和多方签名），远超 12 秒的 slot。一个更现实的预期是：系统不是用来在**同一区块内**阻止攻击的，而是用来在攻击上链后的**第一时间**（亚秒级）发出告警，配合协议的暂停机制或保险基金来限制损失。这与 DeFiGuard 等事后方法的区别在于：本文的系统在攻击交易进入 mempool 的瞬间即可开始检测流程，而非等待交易被包含进区块后才启动。

7.5 伦理与合规考量

第三方标签的使用边界：系统使用 `[LABEL:provider:confidence]` 格式引入地址标签（如 SlowMist、MistTrack），但明确规定 LABEL 证据不能作为确定性身份结论。这不仅是技术设计选择，也是法律合规要求——在多数司法管辖区，将链上地址与真实身份关联需要经过法律程序（如执法请求、法院命令），自动化的标签匹配不能替代。

自动化 vs 人工控制：系统默认不执行自动链上防御。这一设计的伦理基础是"人类在环"（human-in-the-loop）原则——在涉及资金安全的决策中，自动化系统应提供信息和建议，但最终决策权应保留在人类手中。这降低了误报的系统性风险（一次错误的自动防御可能导致协议服务中断和声誉损失）。

数据安全：完整生产部署中，交易 trace 模拟应在自有节点上完成，原始交易 calldata 不发送给第三方 Provider；当前 Provider-only 原型仅使用 Provider 可返回的基础交易信息和轻特征。RAG 检索中的文档匹配在本地完成（向量库本地部署），LLM 调用如需使用外部 API（如 OpenAI），使用经过脱敏的查询文本（仅包含协议名称、函数签名、图结构摘要，不包含具体地址和金额）。

8 总结与展望

8.1 工作总结

本文提出了一种在以太坊交易 pending 阶段完成 DeFi 价格操纵检测的实时预警系统，并实现了 Provider-only 原型。主要工作总结如下：

1. **系统架构**：设计了五层热路径 + 冷路径分离的检测管线。完整方案中，热路径从 pending 交易监听、未上链 trace 模拟、Cash-Flow Graph 构建到风险评分全流程贯通；原型中，Provider-only 降级路径和规则基线评分链路已经完成回放验证。冷路径异步补齐 RAG 证据解释，不阻塞热路径告警。完整 trace 环境下的延迟预算分配为 P95 < 3s、P99 < 8s。
2. **Cash-Flow Graph 增量建模**：在已有工作的 Address/Asset 节点和 Transfer 边基础上，设计 Protocol 节点（编码协议角色与治理属性）、Call 边（编码函数级语义）和 Context 边（编码时序上下文），构成四类节点、三类边的异构图。与

DeFiGuard 的 7 维度逐点对比（见 2.2.3 节）阐明了增量贡献的具体含义；完整 Call/Context 边效果验证待 trace 环境就绪后完成。

3. **双层检测体系**：规则基线包含异常利润、有序预言机读取、闪电贷调用、过度滑点和薄流动性五类信号，是原型的主工作评分器，也是模型退化时的兜底；GraphSAGE 作为完整 trace 数据就绪后的候选在线主模型。三明治和跨协议清算不声称已由当前五条规则完整覆盖。

4. **证据化告警**：设计了统一的 EvidenceRef 三类证据引用目标规范（GRAPH/DOC/LABEL）。当前原型已生成 risk/graph/trace/label 事件级内部引用；路径级强校验和高危告警必备文档证据约束属于生产化待完成项。

5. **端到端实现与验证：以 Uniswap V3、Aave V3、Compound III、Chainlink 为目标协议，完成系统原型实现。已完成的验证：Provider-only 管线完成 1,015 个 replay 工件审计，并在 110 条唯一评分链路上通过 9 项 replay 检查；故障注入实验验证了三级降级协议的健壮性（trace 失败 → 轻特征包、上下文超时或为空 → tx-only 图、模型不可用 → 规则基线），67 个报告文件通过 secrets 扫描；代码层面已补齐 Context 窗口构建、图级标签导出、GraphSAGE 训练入口和推理降级入口。因当前环境限制未执行的部分：GraphSAGE 模型训练（57 笔候选交易已入队但 trace 数据不可用，且尚无 promoted 图级标签）、在线延迟 SLO 实测（需本地节点）、生产故障切换和灰度发布。这些缺口的具体原因和解决方案在 7.1 节和 8.2 节中讨论；模块数量、配置文件和事件 Schema 数量等工程计数列于附录。**

8.2 未来工作

突破 trace 数据瓶颈（若未来扩展为生产级系统）：当前 Provider-only 模式限制了 Cash-Flow Graph 中 Call 边和 Context 边的充分填充，以及 GraphSAGE 的真实训练。若未来扩展为生产级系统，可引入 trace-capable 本地 Geth/Erigon 节点，获取 debug_traceCall 完整 trace 数据。该部分超出当前工作范围。一旦 trace 环境就绪，以下工作可并行推进：57 笔候选交易的 trace-backed 标注推广、GraphSAGE 模型训练、各层真实延迟 SLO 测量、离线对比实验（GCN/GAT/GIN/MLP vs GraphSAGE vs 规则基线）。

多链与 L2 扩展：将系统扩展到 Arbitrum、Optimism、Base 等 Ethereum L2 网络和 Polygon、BNB Chain 等 EVM 兼容链。L2 的特殊性在于 sequencer 的 mempool 策略与 L1 不同，需要针对性适配。

时序图模型二级复核：将 STA-GT 或 FraudGT 集成到冷路径，对 top 1-5% 高风险交易进行二级复核，利用时序模型的长程依赖能力降低误报率。

自动链上防御：在治理框架和权限控制完备后，为系统增加可选的自动防御模块。关键问题包括：防御交易的权限委托（协议多签授予系统有限的 pause 权限）、防御触发条件（仅在 score > 0.95 + 至少 3 个独立信号同时触发时才自动执行）、可逆性（自动执行的防御交易必须可被多签撤销）。

跨协议协同检测：将多笔邻近交易合并到同一个时序图中，检测跨交易的协同操纵行为。

与 MEV 基础设施集成：与 Flashbots 等 MEV 基础设施合作，获取私有 mempool 和 bundle 数据的访问权限，扩大检测覆盖范围。

联邦学习：在多个协议安全团队之间通过联邦学习共享模型参数（而非原始交易数据），解决标注数据稀缺和隐私保护的双重约束。

参考文献

1. DefiLlama Research. "State of DeFi 2025." DefiLlama, 2025. https://defillama.com/research/report/state-of-defi-2025
2. SlowMist. "2025 Blockchain Security and AML Annual Report." SlowMist, 2026. https://www.slowmist.com/report/2025-Blockchain-Security-and-AML-Annual-Report%28EN%29.pdf
3. CertiK. "Hack3d: The Web3 Security Report 2024." CertiK, 2025. https://www.certik.com/resources/blog/hack3d-the-web3-security-report-2024
4. Rekt News. "Mango Markets Exploit: Oracle Price Manipulation Leads to $116M Loss." October 2022. https://rekt.news/mango-markets-rekt/
5. Rekt News. "Cream Finance Flash Loan Attack." October 2021. https://rekt.news/cream-rekt-2
6. Wu, Siwei, Dabao Wang, Jianting He, Yajin Zhou, Lei Wu, Xingliang Yuan, Qinming He, and Kui Ren. "DeFiRanger: Detecting Price Manipulation Attacks on DeFi Applications." arXiv:2104.15068, 2021.
7. Wang, Dabao, Bang Wu, Xingliang Yuan, Lei Wu, Yajin Zhou, and Helei Cui. "DeFiGuard: A Price Manipulation Detection Service in DeFi using Graph Neural Networks." arXiv:2406.11157, 2024.

8. CertiK. "Skynet Security Platform." CertiK, n.d. https://www.certik.com/skynet
9. Hamilton, W. L., Ying, R., and Leskovec, J. "Inductive Representation Learning on Large Graphs." NeurIPS 2017.
10. Tian, Y., and Liu, G. "Transaction Fraud Detection via Spatial-Temporal-Aware Graph Transformer." arXiv:2307.05121, 2023.
11. Chainalysis. "Crypto Crime Reports." Chainalysis, n.d. https://go.chainalysis.com/2025-Crypto-Crime-Report.html
12. Rekt News. "Inverse Finance Price Oracle Manipulation." April 2022. https://rekt.news/inverse-finance-rekt/
13. Daian, P., et al. "Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralized Exchanges." IEEE Symposium on Security and Privacy, 2020.
14. Flashbots. "MEV-Boost Documentation." Flashbots Docs, n.d. https://docs.flashbots.net/
15. Kipf, T. N., and Welling, M. "Semi-Supervised Classification with Graph Convolutional Networks." ICLR 2017.
16. Veličković, P., et al. "Graph Attention Networks." ICLR 2018.
17. Xu, K., et al. "How Powerful are Graph Neural Networks?" ICLR 2019.
18. Dwivedi, V. P., and Bresson, X. "A Generalization of Transformer Networks to Graphs." arXiv:2012.09699, 2020.
19. Lewis, P., et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." NeurIPS 2020.
20. Chen, J., et al. "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation." ACL 2024.
21. Lin, J., Guo, X., Zhu, Y., Mitchell, S., Altman, E., and Shun, J. "FraudGT: A Simple, Effective, and Efficient Graph Transformer for Financial Fraud Detection." In

Proceedings of the 5th ACM International Conference on AI in Finance (ICAIF 2024), pp. 292-300, 2024. https://doi.org/10.1145/3677052.3698648

22. CryptoSlate. "DeFi Lending Protocol bZx Exploit Leads to a $1 Million Loss." CryptoSlate, February 18, 2020. https://cryptoslate.com/defi-lending-protocol-bzx-exploit-leads-to-a-1-million-loss/

23. CertiK. "Euler Finance Incident Analysis." CertiK, March 2023. https://www.certik.com/resources/blog/euler-finance-incident-analysis

附录

附录 A 目标协议合约地址清单

协议	合约角色	Ethereum 主网地址	审计状态
Uniswap V3	Factory	`0x1F98431c8aD98523631AE4a59f267346ea31F984`	audited_recent
Uniswap V3	SwapRouter02	`0x68b3465833fb72A70ecDF485E0e4C7bD8665Fc45`	audited_recent
Uniswap V3	UniversalRouter	`0x3fC91A3afd70395Cd496C647d5a6CC9D4B2b7FAD`	audited_recent
Aave V3	Pool	`0x87870Bca3F3fD6335C3F4ce8392D69350B4fA4E2`	audited_recent
Aave V3	PoolAddressesProvider	`0x2f39d218133AFaB8F2B819B1066c7E434Ad94E9e`	audited_recent
Aave V3	PoolConfigurator	`0x64b761D848206f447Fe2dd461b0c635Ec39EbB27`	audited_recent

协议	合约角色	Ethereum 主网地址	审计状态
Compound III	cUSDCv3 (Comet)	`0xc3d688B66703497DAA19211EEdff47f25384cdc3`	audited_recent
Chainlink	ETH/USD Price Feed	`0x5f4eC3Df9cbd43714FE2740f5E3616155c5b8419`	audited_recent
Chainlink	USDC/USD Price Feed	`0x8fFfFfd4AfB6115b954Bd326cbe7B4BA576818f6`	audited_recent

附录 B 事件 Schema 完整定义

Schema 文件	事件主题	关键字段	用途
`pending-tx.v1.schema.json`	`pending-tx.v1`	`chain_id`, `tx_hash`, `raw_tx`, `seen_at`, `source`	记录首次监听到的 pending 交易
`trace-bundle.v1.schema.json`	`trace-bundle.v1`	`calls`, `logs`, `token_transfers`, `trace_status`, `trace_source`	承载完整 trace 或降级轻特征包
`graph-inference.v1.schema.json`	`graph-inference.v1`	`nodes`, `edges`, `feature_ver`, `feature_missing_flags`	固定 Cash-Flow Graph 推理输入
`risk-score.v1.schema.json`	`risk-score.v1`	score, severity, reasons, rule_scores,	输出风险分数、等

Schema 文件	事件主题	关键字段	用途
		model_version	级和模型来源
alert-event.v1.schema.json	alert-event.v1	alert_id, score, severity, evidence_refs, suggested_actions	面向分析台和通知系统的告警事件
rag-document.v1.schema.json	rag-document.v1	doc_id, source_type, protocol, section, text, source_url	RAG 知识库文档切片索引
rag-explanation.v1.schema.json	rag-explanation.v1	graph_summary, doc_summary, analyst_note, evidence_refs	高危告警的异步证据解释

六个事件 Schema 均包含 event_id、schema_ver、chain_id、created_at 和 idempotency_key；rag-document.v1 为非事件辅助 Schema。事件标识和幂等键由阶段名、交易哈希、来源/版本组合确定性生成。

附录 C 图特征字段字典

对象	字段	类型	Provider-only 状态	完整 trace 状态
Address 节点	account_type, is_contract, in_degree_tx,	categorical / binary / integer	部分可用	完整可用

对象	字段	类型	Provider-only 状态	完整 trace 状态
	`out_degree_tx`			
Address 节点	`call_depth_max`, `profit_path_score`	integer / float	部分可用，依赖轻特征与价格缓存	完整可用
Asset 节点	`token_address`, `symbol`, `decimals`, `asset_type`, `is_stablecoin`	bytes20 / string / categorical	依赖 token metadata cache	完整可用
Protocol 节点	`protocol_id`, `contract_role`, `pool_type`, `audit_status`, `has_pause`	categorical / binary	白名单匹配可用	完整可用
Transaction 节点	`tx_hash`, `from`, `to`, `gas_price`, `value`	bytes / uint256	可用	可用
Transfer 边	`asset`, `amount_raw`, `amount_norm`, `value_usd`, `direction`	bytes / integer / float	部分可用，精确 transfer 依赖日志/trace	完整可用
Call 边	`selector`, `call_type`, `call_depth`, `success`	bytes4 / categorical / integer / binary	仅顶层 selector 可用	完整可用
Context 边	`window`, `relation_type`, `count`, `last_seen_at`	categorical / integer / timestamp	缓存可用时部分可用	完整可用

`feature_missing_flags` 用于显式标记缺失字段，避免训练和推理阶段把"未知"误解为"正常值"。

附录 D 规则基线与核心配置参数

配置文件	核心内容	与规则/告警的关系
`rule_baseline.json`	五条规则的原始权重、阈值、触发强度映射	决定规则基线分数
`reason_codes.json`	`risk_family.signal_name` 原因码、说明、废弃状态	决定告警原因码的稳定语义
`alert_routing.json`	分数区间、severity、通知策略	决定告警路由与处置动作
`protocol_roles.json`	合约地址、协议、角色、审计状态、pause 能力	支撑 Protocol 节点和协议白名单
`protocol_log_addresses.json`	需要监听的协议合约日志地址	支撑日志采集和 trace 补充
`selector_dictionary.json`	bytes4 selector 到函数签名映射	支撑轻特征解析和 Call 边语义
`token_metadata_cache.example.json`	token symbol、decimals、稳定币标记、资产类型	支撑金额归一化和 `value_usd` 计算
`message_topics.json`	事件主题与 Schema 版本映射	支撑消息总线与 replay
`rag_explanation_templates.json`	RAG 解释模板与输出字段约束	支撑冷路径证据说明

附录 D 只列与规则、告警和图构建直接相关的核心配置；完整配置与模板文件清单由仓库中的 `ops/reports/artifact-manifest.latest.json` 维护。

附录 E 告警分级与处置矩阵

Severity	Score 区间	触发条件	处置动作	是否进入 RAG
low	< 0.50	无强风险信号或仅单弱信号	仅落盘，用于训练漂移监控和 replay	否
watch	0.50–0.75	多个弱信号或单个中等信号	进入分析台队列，不触发即时通知	可选
high	0.75–0.90	强风险信号或多规则共同触发	生成复核任务和值班通知建议（原型不外发）	是
critical	> 0.90	多个强信号共同触发，或命中高危路径	升级至 on-call 和多签负责人，生成 runbook 步骤	是

当前原型默认不执行自动链上防御；`suggested_actions` 只生成建议步骤和人工复核清单。

附录 F 主要符号与缩写对照表

符号/缩写	含义
DeFi	Decentralized Finance，去中心化金融
GNN	Graph Neural Network，图神经网络

符号/缩写	含义
GraphSAGE	Graph Sample and Aggregate，图采样与聚合
RAG	Retrieval-Augmented Generation，检索增强生成
PMA	Price Manipulation Attack，价格操纵攻击
Mempool	Memory Pool，交易池（pending 交易等待打包的集合）
MEV	Maximal Extractable Value，最大可提取价值
TVL	Total Value Locked，总锁仓价值
EOA	Externally Owned Account，外部拥有账户
AMM	Automated Market Maker，自动做市商
TWAP	Time-Weighted Average Price，时间加权平均价格
SLO	Service Level Objective，服务水平目标
P95/P99	第 95/99 百分位数
G	异构图
V	节点集合
E	边集合